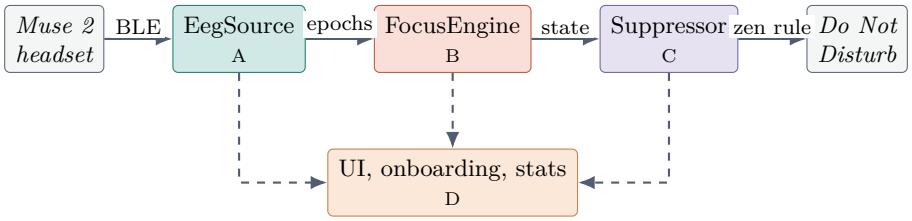
stfu

How the code works

A study guide to the focus-aware notification deferral app



Purpose. Written for the team's internal review before the oral examination. A reader who works through it, with the named source files open beside it, should be able to explain how any part of the app works, why it is built that way, and what its known limits are. Harder parts of the code get more space than easy ones.

Code state. Describes `main` at commit `bf5774e` (3 October 2026), about 9,900 lines of Kotlin, plus the `FocusService` scope fix made while writing this guide. Line references are to that commit and will drift as files change.

Contents

1 How to use this guide 2

2 The system in one picture 2

2.1 Four workloads, one seam 2
2.2 The six contracts 2
2.3 The data classes 2
2.4 The runtime data path 4
2.5 Dependency wiring: `AppContainer.kt` . . . 4
2.6 Threads and scopes 5
2.7 Android platform pieces 6

3 Workload A: sensing and the foreground service 7

3.1 `MuseEegSource`: the production source . . . 7
3.1.1 From packets to an epoch 8
3.2 `ForegroundEegSource`: keep the headset connected while the app is open 8
3.3 The other two sources 9
3.4 `FocusService`: the decision loop 9

4 Workload B: signal processing and calibration 11

4.1 The pipeline per epoch 11
4.2 Artifact rejection 12
4.3 The Schmitt trigger 13
4.4 Drift detection 13
4.5 Calibration: from task blocks to thresholds 14
4.6 Persistence: `ThresholdStore` 15
4.7 Constants to have at hand 15

5 Workload C: notifications 15

5.1 The moving parts 16
5.2 Do Not Disturb: `DndController` 17
5.3 Watching the shade: `FocusNotificationListener` 18
5.4 The queue and the summary 18
5.5 Settings: `NotificationSettings` 18
5.6 What the tests pin down 19

6 Workload D: UI, onboarding, calibration tasks and stats 19

6.1 Structure: one activity, five routes, ViewModels with factories 19
6.2 `LiveStatus` and the home screen 20
6.3 Onboarding 20
6.4 The calibration screen and its tasks 21
6.5 Stats, settings, theme 22
6.6 `SessionStore`: the session log 22
6.7 `ui/preview`: the demo stack 23

7 Integration: what changed when the branches met 23

7.1 Test inventory 24

8 Question bank 24

1 How to use this guide

The app has one job: while a Muse EEG headset says the wearer is concentrating, hold their phone's notifications back, and when concentration breaks, deliver what was held as one quiet summary. Everything in the codebase serves that sentence, and the oral exam will probe whether you can trace any feature back to it.

Reading order. Section 2 explains the shape of the whole thing and the six interfaces that hold it together; read it first regardless of which workload you own. The four workload sections (sections 3 to 6) then go through each area. The length of each is proportional to how hard it is to explain under questioning, not to how many lines it has: the signal path in section 4 gets the most room because it carries the most numbers, and the notification layer gets more than its line count suggests because its behaviour is mostly dictated by Android rules that are easy to misstate. Section 7 covers what changed when the four branches were merged, which matters because several of those changes alter behaviour that a workload owner may still describe the old way. Section 8 is a question bank with answers.

Files. Each section opens with a box naming the source files it describes. The text does not reproduce the code. Open the file, find the function named in the text, and read it against the explanation; that pairing is what the guide is designed for. Line numbers are given as `path:line` and refer to commit `bf5774e`.

Conventions. Workloads are colour-coded in every diagram: A sensing and service, B signal and calibration, C notifications, D UI and stats, S shared core. Dashed teal boxes are interfaces from `core/Contracts.kt`. Focus states are set as FOCUSED, NOT_FOCUSED and UNKNOWN.

2 The system in one picture

Files to read alongside this section

`core/Contracts.kt`, `core/Models.kt`, `AppContainer.kt`, `app/src/main/AndroidManifest.xml`, `DESIGN.md`, `WORKLOADS.md`

2.1 Four workloads, one seam

The project was split four ways before any feature code existed, and the split is visible in the package layout. `WORKLOADS.md` assigns every file to exactly one owner, and the rule was that nobody edits another person's files. The only coupling between owners is the set of interfaces in `core/Contracts.kt` and the data classes in `core/Models.kt`, both written together in "sprint 0" and then frozen. Figure 1 shows the result.

2.2 The six contracts

`core/Contracts.kt` is 122 lines and is the single most important file to know cold. The four original interfaces follow the data path, and two more were added by D during sprint 0 because the UI needed a master switch and a settings surface (the file's header flags them "for ratification").

Why `appliedState` exists alongside `FocusEngine.state`

The engine's state is what the signal says. The applied state is what the suppressor is actually doing, which differs during the 10 s signal-loss grace period and whenever the service is not running. The home screen's status pill shows the applied state so it never disagrees with the Do Not Disturb rule. This member was added during integration; see section 7.

2.3 The data classes

`core/Models.kt` holds every type that crosses a workload boundary. The ones to be able to describe without looking:

`BandPowers`
: Four doubles already reduced from LibMuse's per-channel packets: `alphaTemporal`, `beta1320`, `theta`, `emgBroadband`. The field names encode the design constraints from `DESIGN.md` §1: alpha from the temporal

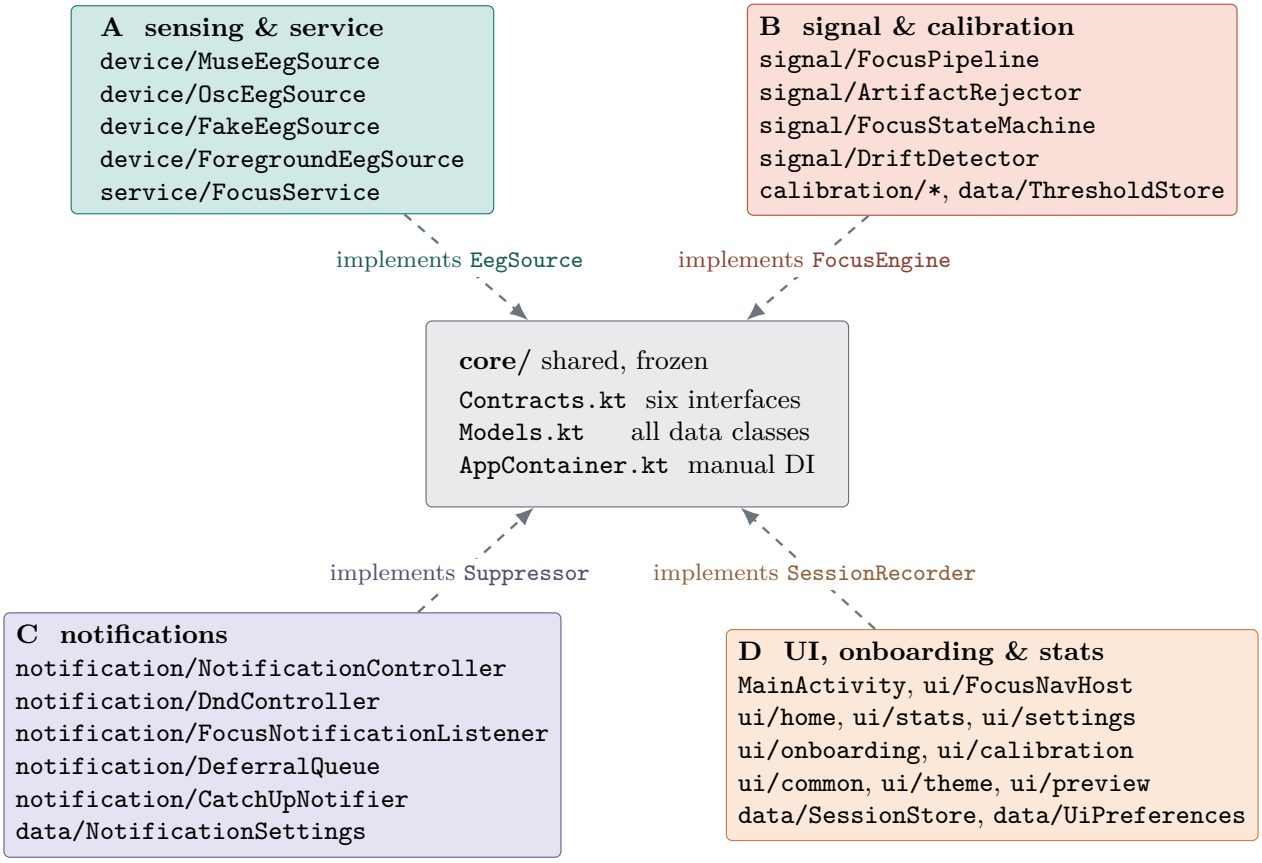


Figure 1: The package layout mirrors the workload split. Each workload implements one of the four original interfaces; the two later additions (`ServiceController`, `NotificationPolicy`) are owned by A and C respectively.

Interface	Owner	Used by	What it promises
`EegSource`	A	B, D	`status: StateFlow<DeviceStatus>`, `epochs: Flow<FocusEpoch>` at about 1 Hz. Emits invalid epochs rather than dropping them, so consumers can tell "bad data" from "no data".
`FocusEngine`	B	A, C, D	`submit(epoch)` in, `state`, `decisions` and `thresholds` out. `thresholds` is `null` until calibrated and consumers must then fail open. Also `reset()`, `updateThresholds`, `fitCalibration`, `recalibrationSuggested`.
`Suppressor`	C	A	`apply(state)` about once a second, so it must be idempotent and only touch the platform on a change. `flush(reason)` releases the queue as one grouped summary. Must fail open on UNKNOWN.
`SessionRecorder`	D	A, C	Session start/end, every decision, every deferral, every flush. Called at 1 Hz from the service scope, so implementations batch their I/O.
`ServiceController`	A	D	`running` and `appliedState`, plus `start()` and `stop()`. The UI must not start the service with a raw Intent because it needs to observe whether the service survived Doze.
`NotificationPolicy`	C	D	`settings: StateFlow<PolicySettings>` and `update()`. Backs the notification section of Settings.

Table 1: The six interfaces in `core/Contracts.kt`.

pair TP9/TP10 because it is less eye-contaminated than the frontal pair, and beta clamped to 13–20 Hz because at a forehead electrode 13–30 Hz is substantially frontalis and temporalis muscle. The computed property `engagementIndex` is the Pope et al. (1995) index $\beta/(\alpha+\theta)$.

`ContactQuality`
Horseshoe indicator (HSI) per electrode, 1 good to 4 no contact. `allGood` requires every electrode to be exactly 1.

`ArtifactFlags`
One boolean per rejection rule: blink, jaw clench, movement, broadband EMG.

`FocusEpoch`
One analysis window: timestamp, `focusIndex`, `bands`, `contact`, `artifacts`, and `valid`. Invalid epochs are still emitted so the UI can draw a gap instead of interpolating.

`FocusState`
UNKNOWN, FOCUSED, NOT_FOCUSED. UNKNOWN means "no usable signal" and always means suppression is off.

`FocusDecision`
The engine's output per epoch: `state`, `rawIndex`, `normalizedZ`, `accepted`. When `accepted` is false the epoch fed no decision and `normalizedZ` is meaningless.

`FocusThresholds`
T_{high} and T_{low} as z-scores, `entryDwellSec`, `exitDwellSec`, `fittedAtMs`, `auc`. The `PLACEHOLDER` set, never fitted to anyone, is $+0.5$, -0.2, 10 s and 60 s.

`CalibrationBlock, TaskScore, CalibrationResult`
The B/D seam. D's task screens produce blocks with a `TaskScore`; B's fitter consumes them and returns a result whose `fitted` flag distinguishes "not enough clean data" from "poor separation".

`DeferredNotification, FlushReason, PolicySettings`
The notification side. `FlushReason` has five values so the stats screen can tell the safety valves apart: `FOCUS_BROKE`, `MAX_HOLD_EXCEEDED`, `HEADSET_REMOVED`, `SERVICE_STOPPED`, `MANUAL`.

`DeviceState, DeviceStatus`
Connection state plus battery and contact. `usable` is true only for `CONNECTED`; `POOR_CONTACT` is connected but not trustworthy.

`SessionSummary, SessionStats`
What the stats screen renders. Focused time is kept per session as a 24-slot by-hour list so the time-of-day pattern survives deleting old sessions.

Why thresholds are z-scores and never raw μV^2

Absolute band power drifts with electrode placement, hair, skin moisture, headband tension, time of day and caffeine. A raw number calibrated on Tuesday is wrong on Thursday. Storing z-scores makes the thresholds relative to a distribution, and the pipeline's job becomes supplying the right distribution to score against (section 4.1).

2.4 The runtime data path

Figure 2 is the picture to have in your head when answering any "what happens when…" question. The headset produces samples at 256 Hz; LibMuse reduces them to band powers; the source reduces those to one `FocusEpoch` per second; the engine turns epochs into a state; the suppressor turns state changes into Do Not Disturb toggles; and the recorder logs the lot for the stats screen. The whole chain runs inside `FocusService`'s coroutine scope regardless of whether any screen is open.

2.5 Dependency wiring: `AppContainer.kt`

There is no DI framework. `FocusApp`, the `Application` subclass named in the manifest, constructs one `AppContainer` in `onCreate()`, and everything else reaches it through `(application as FocusApp).container`. The container's header gives the reason: five singletons do not justify a build dependency, and a container the whole team can read on one screen is worth more than annotations.

The container builds, in order:

- `applicationScope`: `CoroutineScope(SupervisorJob() + Dispatchers.Default)`. It outlives every ViewModel because the decision loop must keep running with no screen bound.
- D's `UiPreferences` and `SessionStore`; the latter is also exposed as the `SessionRecorder`.
- A's `eegSource = ForegroundEegSource(MuseEegSource(context))`. The comment notes that `OscEeg-`

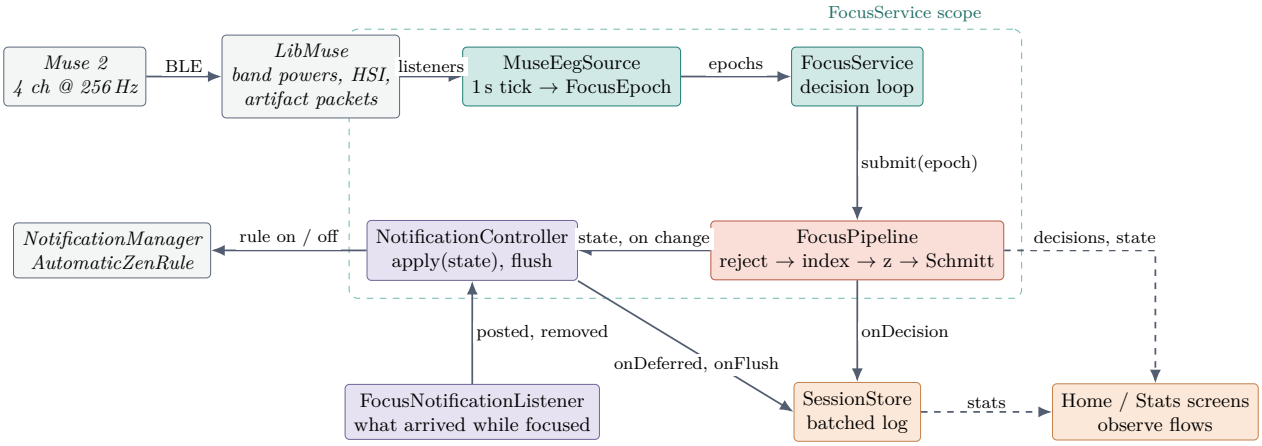


Figure 2: The runtime data path. Everything inside the dashed outline runs in the foreground service's coroutine scope; the UI only observes.

`Source` and `FakeEegSource` are drop-in replacements for development; swapping the source is a one-line edit here, nowhere else.

- A's `AndroidServiceController`.
- B's `ThresholdStore` and `FocusPipeline(store, scope, smoothingWindow = 8)`.
- C's `NotificationSettings` (the `NotificationPolicy`) and the `NotificationController` assembled from `DndController`, `DeferralQueue`, `CatchUpNotifier`, the recorder, the policy and the scope.

Why the smoothing window constant lives in the container

`FOCUS_SMOOTHING_EPOCHS = 8` is a product decision (ride out single noisy epochs, stay well inside the dwell times) rather than a property of the algorithm, so it is set at the wiring point. B's `FocusPipeline` defaults to 1, which keeps B's unit tests describing the unsmoothed algorithm.

2.6 Threads and scopes

The question "which thread does this run on" comes up for every workload, so it is worth collecting the answer in one place (fig. 3).

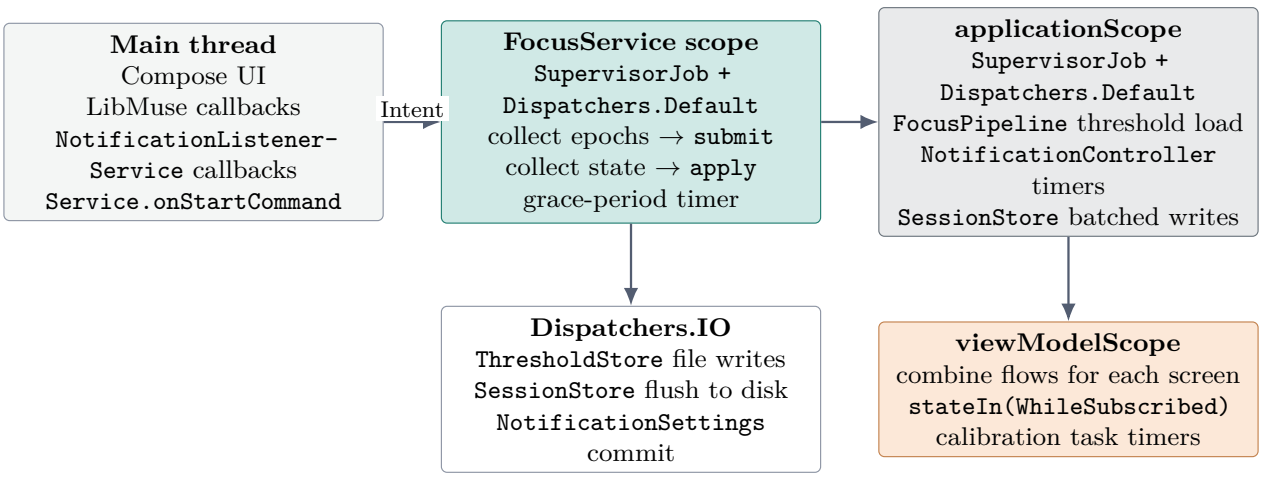


Figure 3: Where code runs. Shared mutable state between these is guarded either by a `StateFlow` (the normal case) or by an explicit lock (`FocusPipeline.submit/reset`, `DeferralQueue`, `NotificationController`'s mutex, `DndController`).

The rule of thumb is that platform callbacks arrive on the main thread, every long-lived loop lives in a `SupervisorJob` scope on `Dispatchers.Default`, and the only places that touch disk hop to `Dispatchers.IO`. State crosses between them almost exclusively as `StateFlows`, which is why nearly every class in the project exposes one.

2.7 Android platform pieces

`AndroidManifest.xml` was written in sprint 0 so that the four workloads never raced on it. It declares three components and the permissions in table 2.

- `FocusApp` as the `Application` class.
- `MainActivity`, the single activity hosting the Compose navigation graph. It also carries the `AUTOMATIC_ZEN_RULE` intent filter and the `ruleType`/`ruleInstanceLimit` meta-data, because the Do Not Disturb rule names it as its configuration activity.
- `service.FocusService` with `foregroundServiceType="connectedDevice"`, not exported.
- `notification.FocusNotificationListener`, exported, guarded by the system-only `BIND_NOTIFICATION_LISTENER_SERVICE` permission, with the `NotificationListenerService` intent filter.

Permission	Kind	Why
`BLUETOOTH_SCAN` (`neverForLocation`)	runtime, 31+	Find the headband. Without the flag, Android 12+ silently drops every scan result unless fine location is also held.
`BLUETOOTH_CONNECT`	runtime, 31+	Connect to it.
`ACCESS_FINE_LOCATION` (`maxSdk 30`)	runtime	Pre-31 equivalent for BLE scanning.
`BLUETOOTH`, `BLUETOOTH_ADMIN` (`maxSdk 30`)	install	Legacy, still required by some LibMuse-internal calls.
`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_CONNECTED_DEVICE`	install	Host the BLE link and decision loop; Android 14+ requires the typed permission.
`POST_NOTIFICATIONS`	runtime, 33+	The service notification and the catch-up summary.
`ACCESS_NOTIFICATION_POLICY`	special access	Create and toggle the `AutomaticZenRule`.
`REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`	special access	Ask to be exempt from Doze and OEM battery killers.
Notification listener access	special access	Granted in system Settings, not via a dialog. Lets the listener see what arrived while focused.

Table 2: Permissions declared in the manifest and requested by D's onboarding flow.

The `neverForLocation` flag was the first headset bug

The app originally scanned forever and `museListChanged()` never fired, although the phone could see `Muse-3973` advertising. `BLUETOOTH_SCAN` without `usesPermissionFlags="neverForLocation"` only receives results when the app also holds `ACCESS_FINE_LOCATION`, which is capped at SDK 30 here and so can never be granted on a modern phone. LibMuse's `startListening()` swallows the exception, so nothing surfaced. The fix is one attribute in the manifest. The full account is in `MUSE_CONNECTION_FIXES.md` §1.

Build. `app/build.gradle.kts` targets SDK 37 with `minSdk = 26`, Kotlin 2.2 with the Compose compiler plugin, and Material 3 through the Compose BOM. LibMuse is a plain `libmuse_android.jar` in `app/libs`, picked up by the `fileTree` dependency. The only non-Compose libraries are Navigation Compose, DataStore Preferences and `kotlinx-coroutines-test`. Each workload owns its own `strings_*.xml` so the resource files never conflict. Gradle needs `JAVA_HOME` set, and the emulator is not usable on the development machine, so testing has been on a physical phone over wireless `adb`.

3 Workload A: sensing and the foreground service

Files to read alongside this section

`device/MuseEegSource.kt`, `device/ForegroundEegSource.kt`, `device/OscEegSource.kt`, `device/FakeEegSource.kt`, `service/FocusService.kt`, `MainActivity.kt`, `MUSE_CONNECTION_FIXES.md`, `docs/area_specific_changes.md` (Workload A)

Workload A owns two things: getting a usable `FocusEpoch` out of the headset once a second, and the foreground service that keeps the decision loop alive with no screen open. The first is mostly a fight with LibMuse's undocumented behaviour; the second is mostly a fight with Android's power management. Both are documented in the two markdown files above, which are worth reading in full because they record what was observed on the phone, not just what was changed.

3.1 `MuseEegSource`: the production source

The class wraps Interaxon's LibMuse SDK (a plain `libmuse_android.jar` in `app/libs` plus native libraries under `jniLibs`). It subscribes to LibMuse's pre-computed relative band powers, the horseshoe contact indicator, accelerometer, battery and artifact packets, and once per second reduces them to one epoch. It also owns the whole scan, connect, drop and rescan lifecycle, shown in fig. 4.

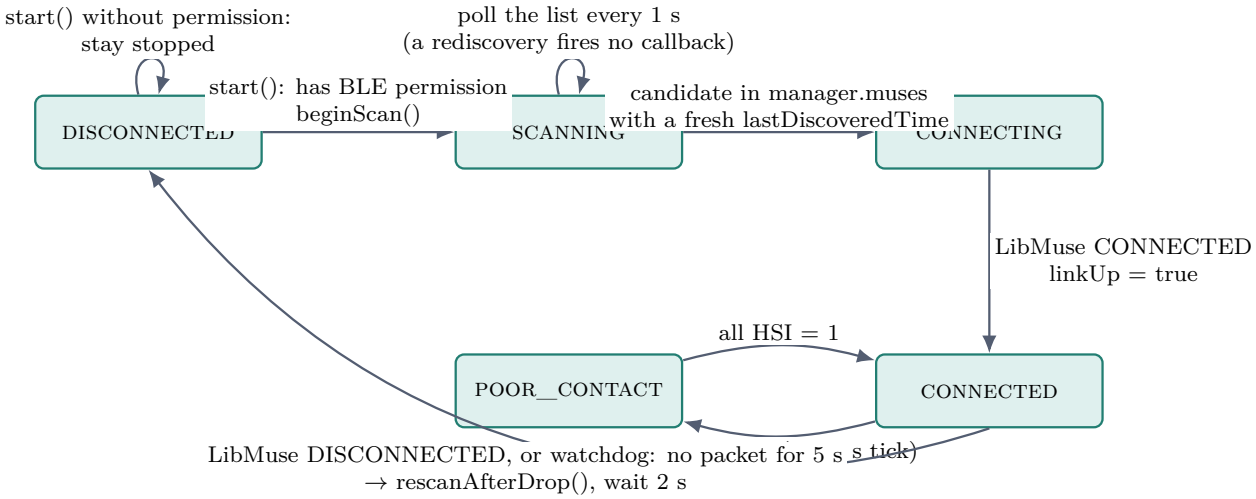


Figure 4: The Muse source's connection lifecycle as the rest of the app sees it through `DeviceStatus.state`. Only CONNECTED counts as `usable`.

Starting (`MuseEegSource.kt:89`). `start()` is idempotent via a `stopped` flag. If the Bluetooth runtime permissions are missing it logs, publishes DISCONNECTED and returns *without* clearing `stopped`, so the next `start()` after onboarding grants access really scans. The earlier behaviour set the flag with no scan running, and every later start returned early until the process was killed. Otherwise it installs the LibMuse listener, calls `beginScan()` and launches the tick and watchdog loops.

Finding the headband. LibMuse only fires `museListChanged()` for a headband it has never seen before. One already in its internal list, which is the normal case after a drop or an app restart, is silently "rediscovered" with a timestamp update and no callback. So `beginScan()` calls `connectToAvailableMuse()` directly after `startListening()` and then polls the list every second until something connects. The selection is `@Synchronized` because it is reached both from the main-thread callback and from the polling coroutine.

Not reconnecting to a ghost. After a drop LibMuse keeps the headband listed for about 30 s with its old discovery time. Connecting to that entry while the headband is switched off just times out, showing "Connecting" the whole while. `rescanAfterDrop()` therefore records the headband's `lastDiscoveredTime` as the stale time, and the selection in `connectToAvailableMuse()` only takes an entry whose time is newer, meaning the headband is genuinely advertising again.

Connecting (`MuseEegSource.kt:196`). `connectTo()` resets the per-connection band state, registers listeners for exactly eight packet types (the four `*_RELATIVE` bands, `ACCELEROMETER`, `HSI_PRECISION`, `BATTERY`, `ARTIFACTS`), sets `PRESET_21` and calls `runAsynchronously()`. LibMuse's connection listener then drives the state: `CONNECTED` sets the `@Volatile linkUp` flag and stamps `lastPacketAtMs` so the watchdog does not fire before the first packet; `DISCONNECTED` triggers a rescan after 2 s; `NEEDS_UPDATE` and `NEEDS_LICENSE` drop to DISCONNECTED with no retry.

Drop detection is belt and braces: LibMuse's own `DISCONNECTED` callback, plus a watchdog that treats 5 s without any data packet while CONNECTED or POOR_CONTACT as a drop. Both paths go through `rescanAfterDrop()`.

3.1.1 From packets to an epoch

LibMuse callbacks arrive on the main thread, so the per-tick accumulators are guarded by `synchronized(lock)`. The data listener stores each band packet through `mergeFresh()` (`MuseEegSource.kt:273`), which is the single most important hack in the file.

Why `mergeFresh` compares every channel against the previous packet

For the relative band packets LibMuse sends `NaN` for a channel until it has its first clean window, and afterwards *repeats the last clean value* while it rejects the current window for a blink or poor contact. Averaging all four channels meant one `NaN` channel made the whole index `NaN`, and repeated values were treated as fresh data; in one session the frontal alpha did not change for three and a half minutes. `mergeFresh` keeps only per-channel values that are finite and differ from the previous packet, and a channel with nothing fresh this tick stays `NaN`. The first focus value then appeared about a second after contact settled, instead of about ninety.

Every `periodMs` (1 s) `emitEpoch()` (`MuseEegSource.kt:320`) snapshots and clears the accumulators, and then:

1. Returns without emitting if there is no active headband or `linkUp` is false. Deriving state from an empty horseshoe during a reconnect is what used to show "Poor contact" instead of "Connecting".
2. Publishes CONNECTED if every electrode's HSI is 1, otherwise POOR_CONTACT.
3. Reduces the bands: `alphaTemporal` is the mean of channels 0 and 3 (TP9 and TP10), `beta1320`, `theta` and `emgBroadband` are four-channel means of LibMuse's beta, theta and gamma. The averages skip `NaN` channels and return `NaN`, not 0, when none remain, so missing data can never produce a plausible-looking index.
4. Sets `ArtifactFlags` from the sticky blink and jaw-clench flags, from an accelerometer delta norm above 0.6, and from an EMG rule `gamma > 3.0 && beta > 3.0`.
5. Sets `valid` to true only if no flag is set, contact is all good and the index is finite, then `tryEmit`s the epoch on a shared flow with an 8-slot buffer.

Three things to say honestly if asked

The field names in `BandPowers` promise 13–20 Hz beta and 20–40 Hz EMG, but the Muse source fills them with LibMuse's whole beta band and its gamma band; the clamp is a contract, not something this source enforces. LibMuse's relative band powers are fractions in $[0, 1]$, so the EMG rule with thresholds of 3.0 can never fire on this source. And on the phone roughly one epoch in three was invalid for blinks, with AF8 at HSI 2 the next most common cause, because `allGood` demands every electrode at exactly 1.

3.2 `ForegroundEegSource`: keep the headset connected while the app is open

A 35-line decorator using Kotlin interface delegation (`EegSource by inner`) that overrides only `stop()`. `hold()` sets a flag and starts the inner source; `release(keepConnected)` clears the flag and stops it unless a session is running; and while held, `stop()` is ignored. `MainActivity` calls `hold()` from `onResume()` and `release()` from `onStop()`, skipping the release on configuration changes so rotation does not drop BLE.

Why `onResume` and not `onStart`, and why ignore `stop()`

The runtime permission dialog only pauses the activity, and the first real scan has to happen after the grant, so the hold must be re-run on resume. Calibration and `FocusService` both call `stop()` when they finish; honouring it would drop the headset with the app still visible, and before this wrapper existed the headset only connected once calibration started, which ate into the 45 s baseline countdown.

3.3 The other two sources

`OscEegSource` is the prototyping shortcut from `DESIGN.md` §5: the Mind Monitor app streams Muse data as OSC messages over UDP, and this class listens on port 5000 and produces the same epochs. The receive loop runs on `Dispatchers.IO` because `DatagramSocket.receive()` blocks; shared state is behind a coroutine `Mutex` rather than `synchronized` because, unlike LibMuse, all its writers are coroutines. The private `OscDecoder` handles the subset of OSC that Mind Monitor sends: a `/`-prefixed address, a type-tag string, and big-endian `f`, `i`, `T`, `F` and `s` arguments, all 4-byte aligned. Table 3 lists the addresses it understands. Two quirks matter if you swap it in: it publishes POOR_CONTACT one second after start regardless of whether any packet arrived, and it uses Mind Monitor's `_absolute` bands where the Muse source uses LibMuse's `_RELATIVE` ones, so the index and the EMG thresholds are on a different scale.

OSC address	Arguments	Used for
`/muse/elements/alpha_absolute` etc.	4 floats, TP9 AF7 AF8 TP10	the four bands
`/muse/elements/horseshoe`	4 floats, 1/2/4	`ContactQuality`
`/muse/elements/blink`, `jaw_clench`	1 float, > 0	sticky artifact flags
`/muse/acc`	3 floats	movement if delta norm > 0.6
`/muse/batt`	first arg	battery, divided by 10

Table 3: Mind Monitor messages parsed by `OscEegSource.handleMessage`.

`FakeEegSource` is the headset-free source that let B, C and D run the app on day one. It fakes a connection (SCANNING, 400 ms, CONNECTING, 600 ms, CONNECTED at 87 % battery), then emits one epoch per second from a seeded random generator. "True focus" f follows a 180 s raised-cosine cycle, and the bands are linear in f with uniform jitter: $\beta = 3 + 4f$, $\alpha = 4 - 2f$, $\theta = 3 - f$, so the index runs from about 0.43 to 1.75 (fig. 5). Five percent of epochs carry one random artifact flag, and every 97 epochs there is a five-epoch poor-contact episode. It is no longer selectable at runtime: a settings switch existed for about an hour during integration and was removed, so using it means editing the one line in `AppContainer.kt`. Do not confuse it with D's `DemoEegSource` in `ui/preview/DemoStack.kt`, which backs Compose previews.

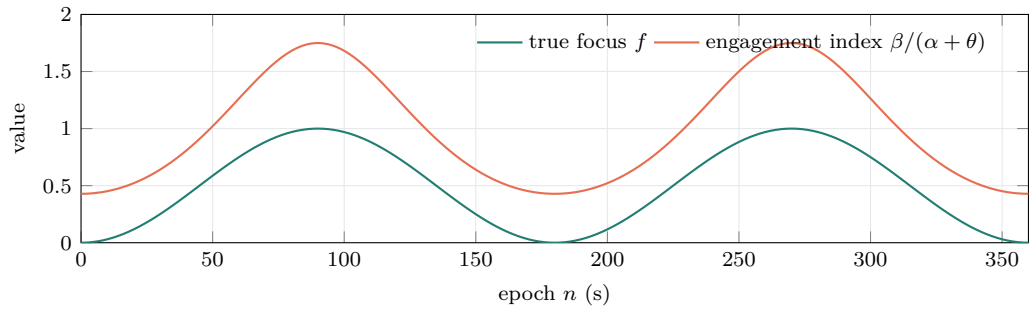


Figure 5: What `FakeEegSource` generates, before jitter and artifacts. Note the cycle ignores whatever calibration block is running, which is why a calibration on the fake source is usually non-separable.

3.4 `FocusService`: the decision loop

The service is declared with `foregroundServiceType="connectedDevice"`, is not exported, and is not bound: `onBind` returns null. It is started and stopped with plain intents through the companion `start(context)` and `stop(context)`, and the UI observes it through two companion-object `MutableStateFlows`, `isRunning`

and `appliedState`, which `AndroidServiceController` exposes as the `ServiceController` contract. If the process dies those flows reset to false and UNKNOWN, which is the honest answer to "did Doze kill it?". `onStartCommand` returns `START_STICKY` so the system recreates the service after killing the process, and because all the wiring is in `onCreate`, a sticky restart restarts the loop.

`onCreate` resolves the container, creates the `focus_service` channel at `IMPORTANCE_LOW` (an app for reducing interruptions must not itself interrupt), calls `startForeground` with notification id 1001 and the connected-device type, logs whether the app is exempt from battery optimisation, and launches `runDecisionLoop()` (`FocusService.kt:97`), which does the following in order:

1. `focusEngine.reset()` and `sessionRecorder.onSessionStart()`.
2. `startWithReconnectBackoff()`: start the source, then watch its status. When it becomes usable, reset the backoff to 2 s; when it drops to DISCONNECTED after having been usable, wait, call `start()` again, and double the wait up to 30 s. For the Muse source this is largely redundant because it rescans by itself, but it matters for a source whose `start()` really reopens something.
3. Collect `epochs` into `focusEngine.submit`.
4. Build `signalUsable`: the status mapped to `usable`, `distinctUntilChanged`, then `mapLatest` so that a drop only becomes "false" after `SIGNAL_GRACE_MS` (10 s) and is cancelled if the signal comes back first.
5. Combine that with `focusEngine.state` into an effective state: the engine's state when usable, otherwise UNKNOWN. On every change: if UNKNOWN and anything is held, `flush(HEADSET_REMOVED)` and `focusEngine.reset()`; then `suppressor.apply()`, publish `appliedState`, and update the service notification text.
6. A second `mapLatest` on usability that calls `stopSelf()` after `STOP_AFTER_SIGNAL_LOSS_MS` (120 s) without signal. A headset that stays away is a session that is over.
7. Collect `decisions` into `sessionRecorder.onDecision`.

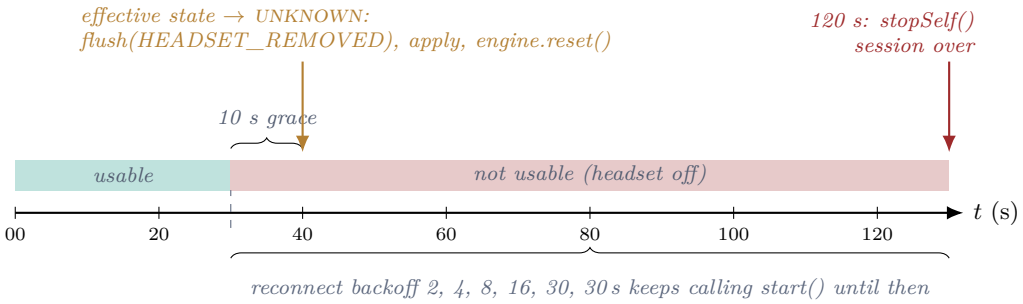


Figure 6: Signal loss in `FocusService`. A blip shorter than the grace period changes nothing; after it the service fails open, and after two minutes it ends the session.

Why the grace period and the reset on loss

On the phone a single electrode reading HSI 2 for four seconds used to release the whole queue and toggle DND off and on. `mapLatest` with a 10 s delay makes a loss count only once it has lasted. The `reset()` on loss means a reconnect has to re-earn FOCUSED through the entry dwell rather than resuming the instant the link is back. The old `watchForHeadsetRemoved()` was removed because it counted emissions of a `StateFlow`, which only emits on change, so it never counted five of anything.

`onDestroy` fails open on the way out: cancel the loop job, stop the source (a no-op while `ForegroundEegSource` is held), set the companion flows to false and UNKNOWN, and then on the scope `apply(UNKNOWN)`, `flush(SERVICE_STOPPED)` and `onSessionEnd()`. There are no wake locks anywhere; the service relies on foreground status, LibMuse's BLE callbacks and the battery-optimisation exemption requested in onboarding.

Why `runDecisionLoop()` is a `coroutineScope` block

Every collector inside it is started with a bare `launch`, so each is a child of the loop coroutine and therefore of `loopJob`; `startWithReconnectBackoff()` is an extension on that same scope for the same reason. Cancelling `loopJob` in `onDestroy` then stops all of them. An earlier version launched them on the service's own `scope` as siblings, so `loopJob?.cancel()` cancelled a coroutine that had already returned and the collectors outlived the service: with the app in the foreground the headset stayed connected, epochs kept feeding the app-wide engine, and the leaked state collector kept calling `apply()` and overwriting `appliedState` after `onDestroy` had set it to UNKNOWN. The service's `scope` is still used for two things only: launching `loopJob`, and the fail-open work in `onDestroy` that must outlive the cancelled loop.

4 Workload B: signal processing and calibration

FILES TO READ ALONGSIDE THIS SECTION

`signal/FocusPipeline.kt`, `signal/ArtifactRejector.kt`, `signal/FocusStateMachine.kt`, `signal/DriftDetector.kt`, `calibration/CalibrationSession.kt`, `calibration/ThresholdFitter.kt`, `data/ThresholdStore.kt`, `SUMMARY_B.md` (partly stale, see section 7), tests under `test/.../signal/`, `calibration/` and `data/ThresholdStoreTest.kt`

This is the research contribution and the part of the codebase with the most numbers in it. It is almost all pure Kotlin with no Android dependency, which is why it carries about a hundred unit tests and why it is the easiest workload to reason about from the tests alone. Everything in this section is reachable from one method, `FocusPipeline.decide()`, so that is where to start reading.

4.1 The pipeline per epoch

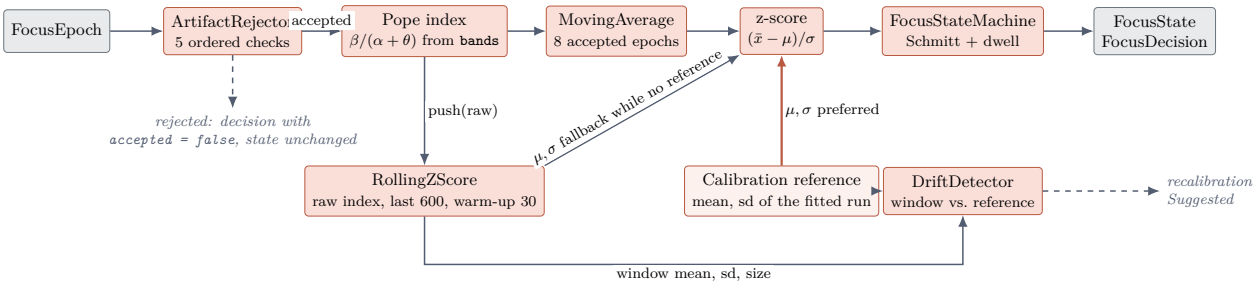


Figure 7: `FocusPipeline.decide()`. The smoothed index is scored against the calibration run's distribution when one exists; the rolling window of *raw* values runs underneath for drift detection and as the fallback.

`submit(epoch)` (`FocusPipeline.kt:109`) takes a lock, runs `decide()`, and `tryEmit`s the resulting `FocusDecision` on a 128-slot shared flow. `decide()` does, in order:

1. Recompute the index from `epoch.bands.engagementIndex`, never trusting `epoch.focusIndex`, so the definition lives in one place whichever source made the epoch.
2. Ask the rejector. A rejected epoch yields a decision with `accepted = false`, `normalizedZ = 0` and the current state, and touches nothing else.
3. `smoother.push(index)`: the mean of the last 8 accepted raw indices (the mean of what is there until the window fills; no warm-up).
4. Pick the reference: the calibration run's mean and SD if `useCalibrationReference` is on, a reference exists, and its SD exceeds 10^{-9}.
5. `rollingZ = normalizer.score(smoothed)`, then `normalizer.push(index)`. The window holds the *raw* index and scores before adding.
6. $z = (\text{smoothed} - \mu_{\text{ref}})/\sigma_{\text{ref}}$ if there is a reference, otherwise `rollingZ`, which is null during the 30-epoch warm-up.
7. If z is null or there are no thresholds, return `accepted = false`. The smoother and window were still fed, so an uncalibrated pipeline is warm the moment thresholds arrive.
8. `_state.value = stateMachine.update(z, timestamp, thresholds)`.

9. `drift.check(window mean, sd, size, thresholds.fittedAtMs)`; if true, latch `recalibrationSuggested`.
10. Return the accepted decision carrying the raw index and z.

Why score against the calibration run and not the rolling window

This is open question 1 in `SUMMARY_B.md` and the most important change made at integration. A 10-minute rolling window re-centres on whatever the user has been doing, so during a long focus period the z-score drifts back to zero and the user drops out of FOCUSED while still concentrating. The thresholds were fitted against the calibration run's distribution, so that is what live samples are now scored against. Replaying 620 epochs from a real reading session with the owner's fitted thresholds gave 0 % of the time FOCUSED with the rolling window and no smoothing, 87 % with the calibration reference, and 98 % with the reference plus 8-epoch smoothing. The rolling window still runs on raw values underneath so the drift detector sees exactly what it did before.

Why smooth, and why keep the window raw

At 1 Hz the raw index is noisy enough that a single dip into the dead band resets the entry run, so "sustained above T_{high} for 10 s" was unreachable: the longest unbroken run in an 11-minute real session was 8 s. Averaging 8 accepted epochs fixes that and stays well inside the dwells. The window is fed the raw value because smoothing shrinks the SD, which would look like spread drift against the calibration reference. The reported `normalizedZ` is the smoothed one so the home chart matches the thresholds drawn on it.

`RollingZScore` (`FocusPipeline.kt:240`) keeps the last 600 values (about 10 minutes) and refuses to score until it has 30. It uses the sample SD ($n-1$), returns 0 for a flat window instead of dividing by zero, and scores a value *before* adding it so a sudden excursion is not partly cancelled by its own contribution. It is a separate class so the fitter can normalise exactly the same way.

The other methods. `updateThresholds` saves to the store *before* applying, so an invalid set is refused by the store's validation rather than half-applied; it leaves the drift reference alone, so editing a dwell time keeps drift detection working. `reset()` clears the rejector, state machine, window, smoother and the drift counter and sets the state to UNKNOWN, but keeps the thresholds and does not clear `recalibrationSuggested`, because the stale thresholds are still the ones in use next session. The startup load uses `compareAndSet(null, loaded)` so a calibration that lands before the asynchronous disk read finishes is not overwritten, and restores the drift reference from the history record whose `fittedAtMs` matches the current thresholds.

4.2 Artifact rejection

`ArtifactRejector.check(epoch)` (`ArtifactRejector.kt:57`) returns null to accept or a `RejectReason`. The checks run in the order of fig. 8, and the first hit wins.

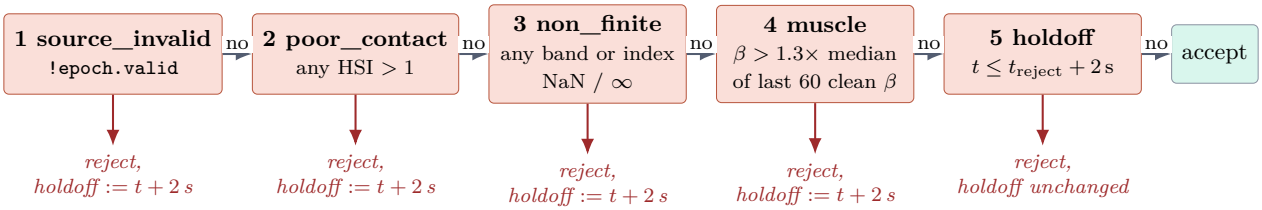


Figure 8: The rejector's ordered checks. A primary rejection (1–4) restarts the 2 s holdoff; a holdoff rejection does not extend it, or one blink would shut the gate forever.

Details that come up in questions:

- The holdoff is inclusive of its end point: an artifact at t also rejects $t+1$ and $t+2$, and $t+3$ is accepted. It exists because artifacts smear across overlapping analysis windows.
- The muscle rule is relative to the user's own recent beta because absolute band power varies between people and sessions. It is skipped until 10 clean epochs exist, and the median is taken *before* adding the current epoch.

- Muscle-rejected epochs still enter the median window. Excluding them would lock the gate permanently if baseline beta genuinely rose; with a one-minute window a brief clench cannot move the median, while a lasting shift is absorbed within about 60 epochs.
- `reset()` clears both the median window and the holdoff.

The muscle rule may reject real focus

If concentrating genuinely raises 13–20 Hz beta by more than 30 % over the recent median, the start of every focus period is thrown away as muscle. A model on the fake source's signal rejected 24 % of epochs as muscle and 42 % overall. This is open question 2 in `SUMMARY_B.md` and is unchanged. Note also that A's sources set their own `valid` flag with their own EMG rule, so two layers of artifact logic overlap (open question 3).

4.3 The Schmitt trigger

`FocusStateMachine.update()` (`FocusStateMachine.kt:44`) takes a z-score, its timestamp and the current thresholds, and is the two-threshold state machine from `DESIGN.md` §1. It is fed only accepted z-scores, and the thresholds are passed in on every call so a changed dwell applies to the run already in progress.

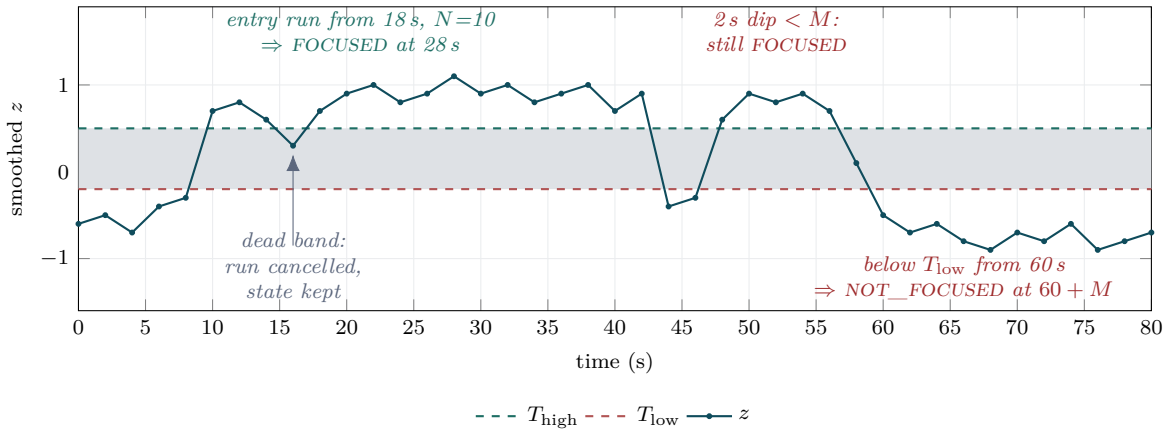


Figure 9: The Schmitt trigger with dwell. Defaults before calibration are $T_{\text{high}} = +0.5$, $T_{\text{low}} = -0.2$, $N = 10\,\text{s}$, $M = 60\,\text{s}$; a fitted set replaces the thresholds, the dwells stay user settings.

The algorithm per call:

1. If more than `maxGapMs` (5 s) has passed since the last sample, discard both runs: a gap means the signal was lost, and nothing is known about it.
2. Classify: $z > T_{\text{high}}$ starts or continues the high run and clears the low run; $z < T_{\text{low}}$ does the opposite; anything in between (the thresholds themselves included, since comparisons are strict) clears both runs but leaves the state alone.
3. Transition: not FOCUSED and the high run has lasted at least `entryDwellSec` ⇒ FOCUSED; not NOT_FOCUSED and the low run has lasted at least `exitDwellSec` ⇒ NOT_FOCUSED; still UNKNOWN ⇒ NOT_FOCUSED (first usable signal fails open); otherwise unchanged.

"Lasted" is measured between timestamps, `now - runStart >= dwell`, not by counting samples, so at 1 Hz a 5 s dwell is satisfied on the sixth sample. The tests pin the consequences: 120 dead-band values hold FOCUSED; alternating 0.4 and 0.6 around $T_{\text{high}} = 0.5$ never chatters because 0.4 is in the dead band, not below T_{low}; an 8-second dip below T_{low} with a 10 s exit dwell does not end focus.

4.4 Drift detection

`DriftDetector.check(windowMean, windowSd, windowSize, fittedAtMs)` (`DriftDetector.kt:55`) compares the rolling window of raw indices with the calibration run's mean and SD. It refuses to judge unless a reference exists, its `fittedAtMs` matches the thresholds in use (so the developer placeholder is never judged), and the window holds at least 300 samples (about 5 minutes, half the window). Drift means the mean has moved more than one reference SD, or the SD ratio is above 2 or below $\frac{1}{2}$; the spread matters because the thresholds

are z-scores, and a changed spread makes the same z a different raw separation. The condition must hold for 120 consecutive checks, about two minutes, and a single clean check restarts the count. The detector only says "now"; the pipeline latches `recalibrationSuggested` until a separable calibration clears it. The reference itself is not persisted, only rebuilt from the history file at startup, so after a restart it takes roughly seven minutes of drifted signal before the prompt returns.

4.5 Calibration: from task blocks to thresholds

D's calibration screen runs the protocol (nine blocks of 60 s, three per condition, order randomised, with a 45 s baseline and 8 s rests; section 6.4) and collects every epoch. When it is done it calls `FocusEngine.fitCalibration(blocks, epochs)`, and B takes over (fig. 10).

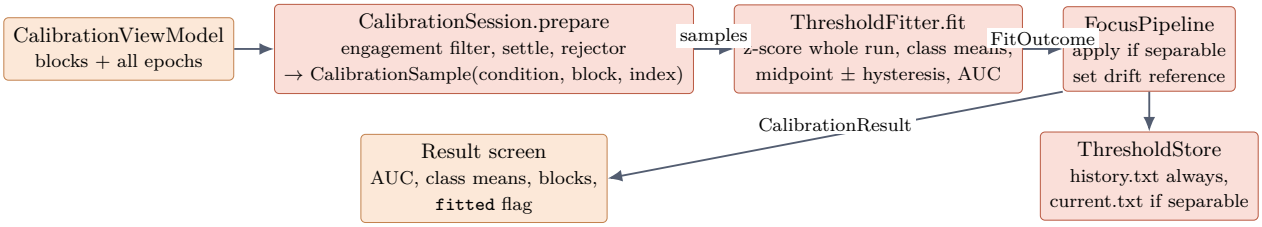


Figure 10: The calibration path. B labels and fits; D runs the tasks and shows the result.

`CalibrationSession.prepare` (`CalibrationSession.kt:63`) turns blocks and epochs into labelled samples:

- **Engagement.** `MEDIUM` and `HIGH` blocks are dropped with a `DropReason` when the task score is missing (`NO_TASK_SCORE`), has zero attempts (`NO_ATTEMPTS`) or accuracy is below 0.5 (`LOW_ACCURACY`; exactly 0.5 is kept). `LOW` blocks have no task and are never dropped. A `HIGH` block where the user disengaged is low-demand data labelled high-demand, the worst input the fitter can get, and random pressing on the 2-back scores about 50 %.
- **Settling.** The first $\min(5\,\text{s}, \text{duration}/12)$ of each block is discarded: a 4 s analysis window plus a second of orienting still contains pre-block signal. The fraction keeps the developer's 6 s fast-mode blocks usable (0.5 s settle).
- **Artifacts.** All epochs, sorted by time, go through one fresh `ArtifactRejector`, so holdoff and the muscle median carry across block boundaries and the fit uses exactly what the live pipeline would have accepted.
- Samples are matched to blocks by wall-clock timestamp (both sides must use the same clock), inclusive at both ends, and tagged with the block's *position* 0–8 rather than its condition so a future leave-one-block-out evaluation can hold out a whole block.

`ThresholdFitter.fit` (`ThresholdFitter.kt:80`) is pure and stateless:

1. Drop non-finite indices. If fewer than 10 `LOW` or 10 `HIGH` samples remain, return `Insufficient`: that is a failed run, not a calibration.
2. z-score every sample, `MEDIUM` included, against the mean and sample SD of the whole run. The nine-minute run is about the length of the live window, so these z-scores sit on the same scale as live ones; `MEDIUM` widens the reference distribution without touching the class means or the AUC.
3. Class means $\bar{z}_{\text{low}}$, $\bar{z}_{\text{high}}$; midpoint $m = (\bar{z}_{\text{low}} + \bar{z}_{\text{high}})/2$.
4. Half gap $h = \max\big(0.15\,(\bar{z}_{\text{high}} - \bar{z}_{\text{low}}),\ 0.05\big)$; $T_{\text{high}} = m + h$, $T_{\text{low}} = m - h$. The dead band is 30 % of the class separation and never narrower than $0.1\,z$, so $T_{\text{low}} < T_{\text{high}}$ holds even for overlapping or reversed classes.
5. AUC by Mann–Whitney: the fraction of (high, low) pairs where the high sample scores higher, ties counting half. It is threshold-free, so it measures the index itself rather than this fitting rule, and 0.5 is chance.
6. `separable = highMean > lowMean && auc >= 0.6`. Dwell seconds pass through unchanged; they are user settings, not fitted.

What the pipeline does with the outcome (`FocusPipeline.kt:161`). `Insufficient` returns a `CalibrationResult` with the thresholds already in use, AUC 0.5 and `fitted = false`; nothing is saved, and the result screen says "not enough clean data" rather than showing an AUC that was never measured. `Fitted` is *always* appended to the history (poor separability is a finding to report, `DESIGN.md` §7), but only made

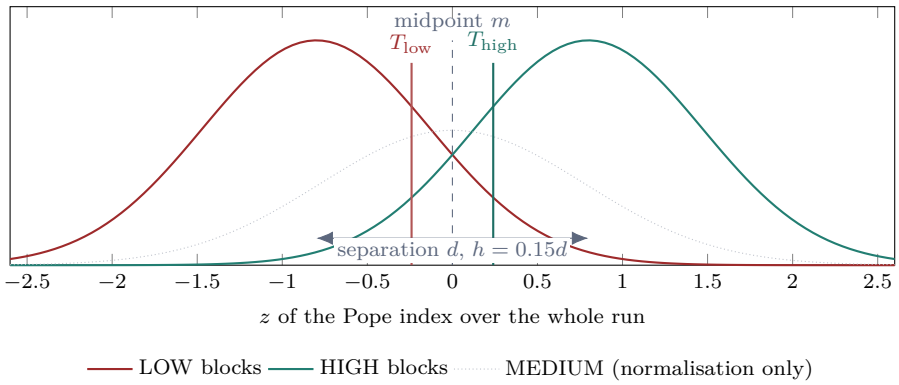


Figure 11: What the fitter computes: thresholds sit ±15 % of the class separation either side of the midpoint between the `LOW` and `HIGH` means. AUC is the probability a random `HIGH` epoch outscores a random `LOW` one.

current, applied, and installed as the drift reference if separable. Acting on an index that does not track this user's engagement would suppress notifications at random, so a non-separable fit fails open. The result always carries every input block, dropped ones included, so the task accuracy shown is honest.

4.6 Persistence: `ThresholdStore`

Two plain-text files under `filesDir/thresholds/`: `current.txt` holds the thresholds in use, `history.txt` gains one line per calibration and is never rewritten. Each line is `key=value` pairs joined by semicolons, for example:

```
tHighZ=0.734;tLowZ=-0.12;entryDwellSec=8;exitDwellSec=45;fittedAtMs=1759000000000;auc=0.81;
    calAuc=0.81;lowMeanZ=-0.6;highMeanZ=0.9;blockCount=9;indexMean=0.42;indexSd=0.07
```

Everything stored is a z-score, a ratio or a count, never a raw band power. `current.txt` is written to a temp file and renamed into place, because the user can change the exit timeout at any time and an interrupted write must not leave half a file. `validate` refuses non-finite thresholds, $T_{\text{low}} > T_{\text{high}}$ and negative dwells. A corrupt history line is skipped; a corrupt current file reads as "not calibrated". Unknown keys are ignored so fields can be added later. All methods take a `Mutex` and run on `Dispatchers.IO`.

Why not JSON or DataStore

`org.json` is a stub on the local JVM, so a JSON store would need Robolectric to test. The key-value format keeps the twelve store tests plain JUnit, and the pipeline tests can use a real store in a temporary directory.

4.7 Constants to have at hand

Table 4 collects the numbers that most often come up when this workload is questioned.

5 Workload C: notifications

Files to read alongside this section

`notification/NotificationController.kt`, `notification/DndController.kt`, `notification/BreakthroughPolicy.kt`, `notification/FocusNotificationListener.kt`, `notification/DeferralQueue.kt`, `notification/CatchUpNotifier.kt`, `data/NotificationSettings.kt`, test `notification/NotificationControllerTest.kt`, `DESIGN.md` §3

The notification layer has the simplest input in the project, a `FocusState` once a second, but the most easily misdescribed behaviour, because most of what happens is done by Android on the app's behalf. The sentence to get right first: *the app never cancels, snoozes or re-posts anyone's notifications.* It switches a Do Not Disturb rule on and off, watches what arrives while the rule is on, and afterwards posts one summary of its own. `DESIGN.md` §3 calls this "v1"; snooze-based true deferral is the unbuilt v2 stretch goal.

Constant	Value	Where
Smoothing window (production)	8 epochs	`AppContainer.kt`
Rolling window / warm-up	600 / 30 values	`RollingZScore`
Muscle ratio / median window / min samples	1.3× / 60 / 10	`ArtifactRejector`
Holdoff after a rejection	2 s, inclusive	`ArtifactRejector`
State machine max gap	5 s	`FocusStateMachine`
Placeholder thresholds	+0.5, −0.2, 10 s, 60 s	`FocusThresholds.PLACEHOLDER`
Drift: mean shift / SD ratio	> 1 ref SD / > 2 or $< \frac{1}{2}$	`DriftDetector`
Drift: min samples / sustain	300 / 120 checks	`DriftDetector`
Settle / fraction / min accuracy	5 s / 1/12 / 0.5	`CalibrationSession`
Hysteresis / floor / min per class / min AUC	0.15 / 0.05 z / 10 / 0.6	`ThresholdFitter`
Real fit on the phone	$T_{\text{high}} = +0.23$, $T_{\text{low}} = +0.07$, AUC 0.67	`area_specific_changes.md`

Table 4: Numbers that appear in questions about workload B.

5.1 The moving parts

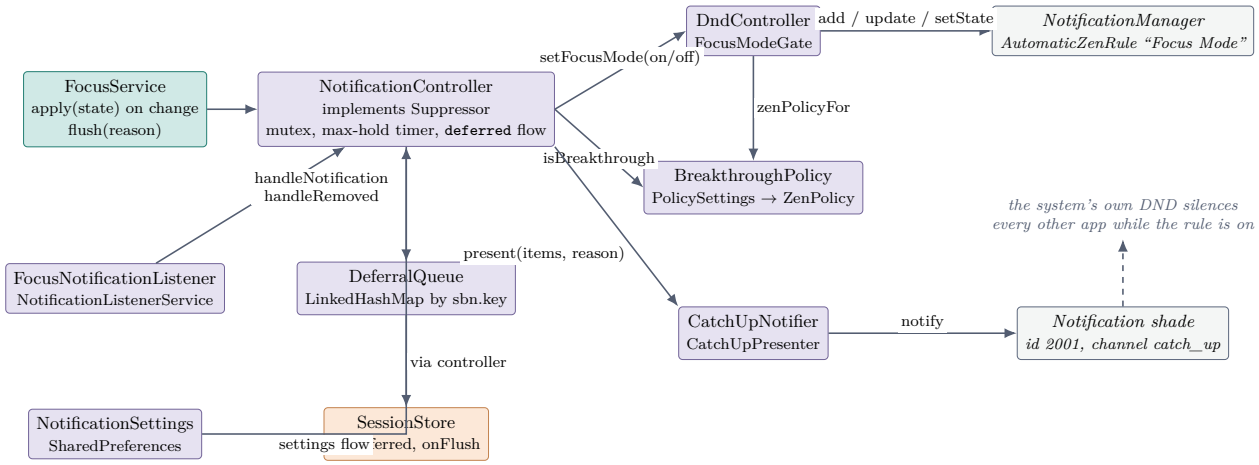


Figure 12: The notification layer. Solid arrows are calls; the two platform objects on the right do the actual suppressing and delivering.

`NotificationController` is the `Suppressor`. It owns a `Mutex` that guards the queue and the max-hold timer together, a `@Volatile currentState` that the listener reads synchronously, and the `deferred StateFlow` the home screen renders. Its constructor takes a `FocusModeGate`, a `DeferralQueue`, a `CatchUpPresenter`, the `SessionRecorder`, the `NotificationPolicy`, a scope and an injectable clock. The two small interfaces exist purely so the controller can be tested with plain JUnit and virtual time.

`apply(state)` (`NotificationController.kt:60`) sets `currentState`, then: FOCUSED turns the gate on; NOT_FOCUSED and UNKNOWN both turn it off and, if anything is queued, call `flush(FOCUS_BROKE)`. The order matters: rule off first, then post the summary, so the summary itself is not silenced by the rule. The service only calls `apply` when the effective state changes (`distinctUntilChanged`), and `setFocusMode` is idempotent anyway, which together honour the contract's "cheap and idempotent at 1 Hz".

`handleNotification(n)` (`NotificationController.kt:71`) returns at once unless focused. Otherwise it copies the notification with `breakthrough` set by `BreakthroughPolicy.isBreakthrough`, launches into the scope, and under the mutex adds it to the queue and republishes. If the queue was empty before the add, it starts the max-hold timer. Outside the lock it calls `recorder.onDeferred`. Breakthrough items are queued and recorded too, just flagged, so the summary and the stats can tell "held" from "delivered during focus".

`flush(reason)` (`NotificationController.kt:99`) cancels the timer, snapshots and clears the queue under the lock, and then, only if something was released, calls `presenter.present` and `recorder.onFlush`. An empty flush is silent: no notification, no stats row.

The max-hold valve (`NotificationController.kt:116`) is a `delay` of the configured minutes followed by a flush with reason `MAX_HOLD_EXCEEDED`. It starts when the queue goes from empty to non-empty and is not reset by later arrivals, so it bounds how long the *oldest* item waits. Focus mode stays on when it fires; only the queue is released. A value of 0 or less disables it. The default is 30 minutes and the settings slider runs 5 to 120.

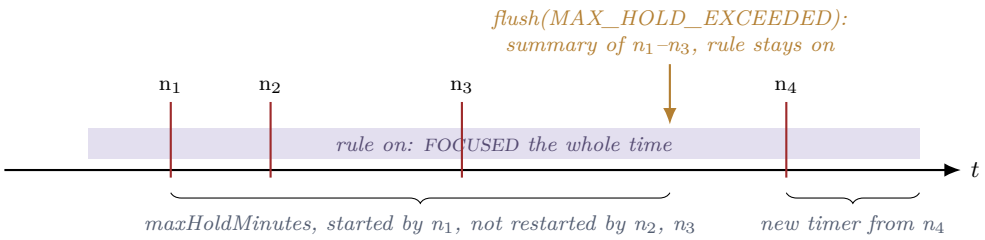


Figure 13: The max-hold safety valve. The timer measures the oldest held item, so a steady trickle of arrivals cannot keep the first one hidden indefinitely.

5.2 Do Not Disturb: `DndController`

The app owns exactly one `AutomaticZenRule`, named "Focus Mode", identified by the condition URI `condition://com.example.compx551a4/focus`. Switching focus on and off never adds or removes the rule; it calls `setAutomaticZenRuleState` with a `Condition` whose state is `STATE_TRUE` or `STATE_FALSE`. The rule's interruption filter is always `INTERRUPTION_FILTER_PRIORITY`, which means "only what the rule's `ZenPolicy` allows".

Three things about `ensureRule` (`DndController.kt:76`) are worth being able to explain:

1. It is **lazy**. The container is built in `Application.onCreate`, before onboarding has granted notification-policy access, so registering in the constructor would fail.
2. It **reuses** an existing rule. It scans `automaticZenRules` for one whose `conditionId` matches before adding; the earlier version added a new rule on every launch.
3. Every platform call is inside `runCatching`, and without policy access the method logs and returns. This is the fail-open behaviour `DESIGN.md` §3 demands: no access, no suppression, notifications arrive normally.

Why a zen rule rather than `setInterruptionFilter()`

A rule is the platform's intended API for app-driven DND. It composes with the user's other rules, appears in system Settings under schedules (where tapping it opens `MainActivity`, which is why the manifest carries the `AUTOMATIC_ZEN_RULE` intent filter), carries its own `ZenPolicy` for the breakthrough list, and the system restores the previous state when the rule turns off. A global filter call would overwrite whatever DND the user had set.

`BreakthroughPolicy` is a stateless `object` with two functions. `zenPolicyFor(settings)` (API 29+) starts from `disallowAllSounds()` and re-allows selectively, so each `PolicySettings` toggle maps one-to-one onto the platform (table 5). `isBreakthrough(n, settings)` is the listener-side classification: true for an allow-listed package, or for category `call` when calls are allowed, or category `alarm` when alarms are allowed. Category is the only signal the listener has; starred-contact and repeat-caller decisions happen inside the system and are invisible to the app.

Known limits to state plainly

`allowedPackages` is flagged in the queue but nothing lets those packages through, and there is no settings UI for it. The old `allowTwoFactorCodes` toggle was removed because nothing could honour it. A starred-contact call that the platform lets through while `allowCalls` is off is flagged `breakthrough = false`. If the service is killed without `onDestroy` running, the rule can stay on until the user turns it off in Settings; the battery-optimisation exemption in onboarding is the mitigation.

PolicySettings	ZenPolicy.Builder call
(always)	`disallowAllSounds()` first
`allowAlarms`	`allowAlarms(bool)`
`allowCalls` / `allowStarredContacts`	`allowCalls(ANYONE / STARRED / NONE)`
`allowRepeatCallers`	`allowRepeatCallers(bool)`
`allowStarredContacts`	`allowMessages(STARRED / NONE)`
`allowedPackages`	nothing: a DND policy has no per-app allowlist
`maxHoldMinutes`	nothing: handled by the controller's timer

Table 5: How `BreakthroughPolicy.zenPolicyFor` translates settings. Below API 29 the legacy rule constructor is used and the system's own Priority settings decide what breaks through.

5.3 Watching the shade: `FocusNotificationListener`

The `NotificationListenerService` is instantiated by the system, so it cannot be constructor-injected; it reaches the controller through `FocusApp.container.suppressor` (`FocusNotificationListener.kt:20`). It overrides only `onNotificationPosted` and `onNotificationRemoved`, and never calls `cancelNotification` or `snoozeNotification`.

`isUserFacing` (`FocusNotificationListener.kt:53`) drops four kinds of notification before they reach the queue: the app's own package (otherwise the catch-up summary would queue itself), ongoing ones (media, navigation, downloads), group summaries (they would duplicate their children), and foreground-service notifications. Everything else that arrives while `controller.isFocused` becomes a `DeferredNotification` keyed on `StatusBarNotification.key`, titled from `EXTRA_TITLE` falling back to `EXTRA_TEXT`, labelled with the app's display name via `PackageManager` (falling back to the package name), and carrying `sbn.notification.category`. Removals forward the key so dismissed items leave the queue.

The manifest entry is guarded by `BIND_NOTIFICATION_LISTENER_SERVICE`, a signature permission that only the system holds. It protects the service from being bound by other apps; it is not something the app requests. The user grants access in system Settings, which onboarding opens with `ACTION_NOTIFICATION_LISTENER_SETTINGS` and checks afterwards with `getEnabledListenerPackages`. In `PermissionChecks.kt` this access is optional: without it DND still silences, the queue just stays empty.

5.4 The queue and the summary

`DeferralQueue` is a `@Synchronized` wrapper around a `LinkedHashMap` keyed on the platform notification key, so it is oldest-first and an app that updates a notification in place ("3 new messages" becoming "4") replaces its entry instead of appearing twice. `add` removes before putting, which moves an updated entry to the newest position. It has no size limit; the max-hold valve bounds it in practice.

`CatchUpNotifier.present` (`CatchUpNotifier.kt:30`) builds the one grouped summary:

- Its own channel `catch_up` at `IMPORTANCE_DEFAULT`, which sounds but never shows a heads-up. `DESIGN.md` is explicit that a burst of heads-up notifications is a worse interruption than the one being prevented.
- Title "n notifications while you were focused". The content line depends on the `FlushReason`: the distinct app names for `FOCUS_BROKE`, otherwise a short phrase such as "Held for the maximum time" or "Headset removed".
- `InboxStyle` with the seven newest items, newest first, as "App: title" lines, and a "$+k$ more" summary line beyond that.
- Tapping it opens the stfu launcher activity, not the originating apps; the originals are still sitting in the shade because DND only silenced them.
- Posted under the constant id 2001, so a second flush replaces the previous summary rather than stacking. A missing `POST_NOTIFICATIONS` grant raises `SecurityException`, which is caught and logged.

5.5 Settings: `NotificationSettings`

A `SharedPreferences` file named `notification_policy`, read once at construction and written on every change. `update` sets the `StateFlow` first so the controller and the settings screen react immediately, then does a synchronous `commit()` on `Dispatchers.IO` so the write is durable before the call returns. The controller's

`init` block collects `policy.settings` and forwards every value to `gate.updatePolicy`, which rebuilds the rule with a fresh `ZenPolicy`. The settings screen edits the four booleans and the slider; nothing edits `allowedPackages`.

5.6 What the tests pin down

`NotificationControllerTest.kt` has nine tests built on fakes for the gate, presenter, recorder and policy, with `runTest` and an unconfined dispatcher so virtual time can be advanced. Being able to recite them is a good way to be sure of the behaviour: FOCUSED turns the gate on and anything else turns it off; nothing is held outside FOCUSED; a focus break releases one summary with reason `FOCUS_BROKE` and the gate is already off when it is posted; an empty queue flushes silently; an updated notification replaces its entry; a dismissed one leaves; max hold fires after exactly `maxHoldMinutes` with focus mode still on, and a later arrival starts a fresh timer; breakthrough items are flagged but still listed; and a policy update reaches the gate exactly once.

6 Workload D: UI, onboarding, calibration tasks and stats

FILES TO READ ALONGSIDE THIS SECTION

`MainActivity.kt`, `ui/FocusNavHost.kt`, `ui/ViewModelFactories.kt`, `ui/model/LiveStatus.kt`, `ui/home/HomeScreen.kt`, `ui/onboarding/*`, `ui/calibration/CalibrationScreen.kt`, `ui/calibration/tasks/*`, `ui/stats/StatsScreen.kt`, `ui/settings/SettingsScreen.kt`, `ui/common/*`, `ui/theme/*`, `ui/preview/*`, `data/SessionStore.kt`, `data/UiPreferences.kt`, tests under `test/.../ui/`

The UI is the largest workload by line count but most of it is ordinary Compose. The parts that need explaining are the ones where a decision was made for a signal-processing or fail-open reason rather than a visual one: the start-destination latch, what the home pill shows, the calibration protocol and its four tasks, and how focused time is measured in the session log.

6.1 Structure: one activity, five routes, ViewModels with factories

`MainActivity` does three things: `setContent` with the app theme and `FocusNavHost`, `eegSource.hold()` in `onResume`, and `eegSource.release(keepConnected = serviceController.running.value)` in `onStop` unless a configuration change is in progress.

Every ViewModel takes plain constructor arguments and gets them through a companion `Factory` built with `viewModelFactory`. The single DI hook is the `container()` extension in `ViewModelFactories.kt`, which casts the application to `FocusApp`. That is why the UI tests need no Android `Context`; the test file header says that if they ever do, something leaked into a ViewModel.

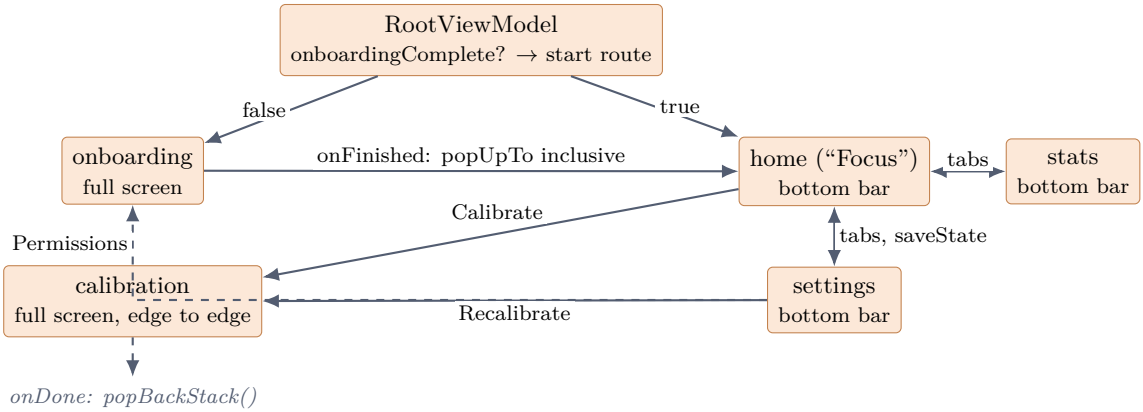


Figure 14: The navigation graph in `ui/FocusNavHost.kt`. Routes are plain strings; the graph has five destinations and no arguments, so type-safe routes were not worth the serialization plugin.

Why the start destination is latched

`RootViewModel` maps the onboarding preference to a start route, with `null` meaning "not read yet". The host renders an empty box until it is known, otherwise the user would land on Home and be yanked to Onboarding a frame later. Once known, the value is latched in `rememberSaveable`, because finishing onboarding flips the preference and letting that reach `startDestination` would rebuild the whole graph under the user. Insets are applied per destination rather than on the `NavHost` because calibration must paint edge to edge.

6.2 `LiveStatus` and the home screen

`liveStatusFlow` in `ui/model/LiveStatus.kt` combines the device status, the *applied* state from the `ServiceController`, the thresholds, the recalibration flag, the deferred list and the running flag into one `LiveStatus`, shared by the home and calibration ViewModels. Its derived `signalTrustworthy` is `device.usable && contact.worst <= 2`.

`HomeViewModel` runs three collectors: the live status (tracking when the focus state last changed), the engine's decisions (kept as the last `CHART_WINDOW = 120` `FocusPoint`s, two minutes at 1 Hz), and an injected one-second ticker that updates the "for 14m 12s" text. The ticker is injected so tests can pass `emptyFlow()`; a `delay` loop inside the ViewModel would hang every test on the virtual clock. Permissions are re-read on every `ON_RESUME` because special access can be revoked in system Settings while the screen is stopped.

Two derived rules decide what the master switch does. `blocker` is `PERMISSIONS_MISSING` if any required permission is absent, else `NOT_CALIBRATED` if there are no thresholds, else null; and `canToggle = serviceRunning || blocker == null`, so a running service can always be turned off. The switch calls `serviceController.start()` or `stop()`; a "release now" button calls `suppressor.flush(MANUAL)`.

The screen is four cards: the switch with a `StatusPill`; the signal card with `FocusIndexChart`; the device card with `ContactQualityMeter`, battery and a contact hint; and the deferred card, which lists "App · count" and never message content, because reading the messages here would be the same interruption the app exists to prevent.

The chart (`ui/common/Charts.kt`) is a hand-drawn `Canvas`, not a library. Its y-range is a fixed ± 3 SD, widened only if the signal leaves it, because auto-scaling would make a calm and a volatile signal look identical. Its x-axis is sized to the *capacity* of 120 slots, so after a screen re-creation new points enter from the right at one per second instead of stretching and then squeezing. Rejected epochs break the polyline, since interpolating over discarded data would misrepresent the one thing the chart shows. The hysteresis band between T_{high} and T_{low} is shaded with dashed guides.

6.3 Onboarding

Five steps, in `PermissionStepId` order: notifications, Bluetooth, listener, DND, battery. Only Bluetooth and DND are marked `required`; the listener is optional because without it DND still silences, you merely get no queue. Each step has a detection rule and a request route (table 6). Results from the launchers are ignored on purpose, and everything is re-read on resume: a denied dialog and a dismissed one are indistinguishable, and the three special-access pages return no result at all. The single footer button reads "Continue" once the required steps are granted and "Skip for now" otherwise; leaving without grants is allowed because fail-open applies to setup too.

Step	Required	Detected by	Requested by
Notifications	no	`checkSelfPermission`	`POST_NOTIFICATIONS` dialog on 33+, nothing below
Bluetooth	yes	`checkSelfPermission`	`BLUETOOTH_SCAN` + `CONNECT` on 31+, `ACCESS_FINE_LOCATION` below
Listener	no	`getEnabledListenerPackages`	`ACTION_NOTIFICATION_LISTENER_SETTINGS`
DND	yes	`isNotificationPolicyAccessGranted`	`ACTION_NOTIFICATION_POLICY_ACCESS_SETTINGS`
Battery	no	`isIgnoringBatteryOptimizations`	direct request with `package:` URI, falling back to the list page on OEMs that refuse it

Table 6: The onboarding steps in `ui/onboarding/PermissionChecks.kt`.

6.4 The calibration screen and its tasks

`CalibrationViewModel` (`CalibrationScreen.kt:127`) is a small state machine over the sealed `CalPhase`, whose cases are Intro, Blocked, Baseline, Countdown, Running, Question, Rest, Fitting and Done. `runProtocol()` (`CalibrationScreen.kt:203`) drives it as in fig. 15.

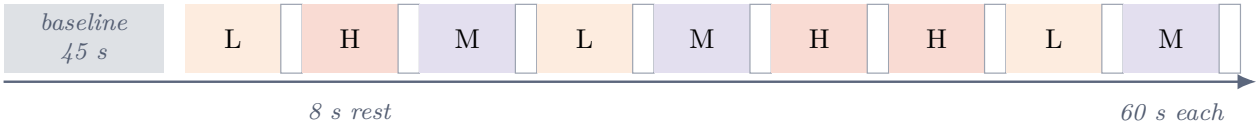


Figure 15: One calibration run, about nine minutes. L, M, H are the `LOW`, `MEDIUM` and `HIGH` conditions, three of each in an order shuffled per run. Before each block the screen waits for a trustworthy signal and counts down from 5. A `MEDIUM` block is followed by one comprehension question. A block with under 50 % valid epochs is discarded and its condition re-queued, at most three times. Fast mode (a developer toggle) uses 6 s blocks, a 4 s baseline, 1.5 s rests, and 600 ms 2-back items.

Points that matter for the fit, all with a comment in the code:

- The source is started by the ViewModel only if no session is running, and `focusEngine.reset()` precedes the baseline. All epochs are captured into one list while `recording` is true; blocks are matched to them later by timestamp.
- Block order is shuffled so a time-into-session effect (fatigue, drying gel, a settling headband) cannot masquerade as a condition effect.
- The reading question is asked *after* recording stops, because answering is a different task and its epochs would be mislabelled `MEDIUM`. Only reading blocks can end early, via "Finished": a finished reader is just waiting. "Not sure" answers null and scores 0 of 1, which B then drops as low accuracy.
- `awaitUsableSignal()` only blocks *between* blocks, the one moment the user can fix the headband without disturbing a recording.
- `accept()` calls `updateThresholds` only if the engine's current thresholds differ from the result's, so a separable fit (already applied by the engine) and an unfitted result (which returns the thresholds already in use) both change nothing.

Matched visual input. Every task renders inside `TaskScaffold`: the same dark `CalibrationTaskTheme` regardless of system theme, the instruction in a fixed 56 dp slot, the task centred in the remaining space, the block label and countdown at the bottom. `DESIGN.md` §2 requires eyes open with matched visual input across conditions; without it the comparison measures screen brightness as much as cognition. For the same reason the screen holds `FLAG_KEEP_SCREEN_ON` and serial subtraction uses an on-screen keypad rather than the IME.

Task	Condition	What happens	Score
`FixationTask`	`LOW`, baseline	A 7 dp dot drifting round a circle, one lap per 20 s, driven by `withFrameNanos` so it still moves with animator scale off. A moving dot holds the gaze and discourages slow blinks.	none
`ReadingTask`	`MEDIUM`	One of three passages, rotated between runs so a repeat is not a memory test, plus "Finished". Then `ReadingQuestion` with three options and "not sure".	0 or 1 of 1
`NBackTask`	`HIGH`	2-back over nine consonants, 1.5 s per item, 30 % targets, never the same letter twice in a row. Hits and correct rejections count as correct; false alarms and reaction times are recorded.	correct / scored
`SerialSubtractionTask`	`HIGH`	Count down in sevens from a random 500–999 on a 4×3 keypad. No right/wrong feedback (an error signal is itself a cognitive event), and the next prompt continues from the true value so one slip does not poison the block.	correct / attempts

Table 7: The calibration tasks in `ui/calibration/tasks/`. The user picks which `HIGH` task to use in Settings.

Two engagement-check weaknesses to know

Because correct non-responses count, a user who never presses MATCH still scores roughly the non-target rate, about 60–70 %, which passes B's 50 % threshold; the check is strongest against random pressing. And a single reading question decides each `MEDIUM` block, so one wrong answer drops it. Both are open questions in `SUMMARY_B.md`.

The result screen distinguishes three outcomes. `fitted = false` shows "Not enough clean data", a dash where the AUC would be (a 0.50 would read as a measured chance result), and offers "Redo" or "Keep current thresholds". A fitted result shows the AUC in large type with a verdict (good at ≥ 0.75, fair at ≥ 0.60, otherwise "barely better than chance", worded as a finding about consumer EEG rather than user error), a `SeparationBar` of the two class means, the thresholds, and the pooled task accuracy, then "Use these thresholds" or "Redo".

6.5 Stats, settings, theme

Stats is `SessionStats` rendered directly: four `StatTile`s (total focus, longest streak, deferred, delivered), a `TimeOfDayChart` of 24 bars where every hour keeps a 2 dp sliver so zero reads as zero rather than a gap, and a history list. There are no queries; all aggregates are getters over the in-memory session list.

Settings writes to three different stores, which is the thing to be clear about (table 8). The dwell sliders are hidden until thresholds exist, because showing them would write placeholder values the user would mistake for their own. "Load placeholder thresholds" is the developer escape hatch that skips calibration.

Setting	Range	Written through
Exit timeout, entry dwell	10–300 s, 3–60 s	`FocusEngine.updateThresholds` → `ThresholdStore` (B)
Max hold, four allow toggles	5–120 min	`NotificationPolicy.update` → `NotificationSettings` (C)
Fast calibration, high-demand task		`UiPreferences` DataStore `ui_prefs` (D)
Placeholder thresholds, clear history		`updateThresholds(PLACEHOLDER)`, `SessionStore.clear()`

Table 8: Where each setting goes. `allowedPackages` has no UI.

Theme. A low-chroma teal and slate palette in `ui/theme/Color.kt`: the app is looked at while concentrating, and during calibration it *is* the visual field. Dynamic colour is off by default so demo screenshots are consistent across devices. Every status colour is paired with an icon and a label, never used alone. `FocusTheme.colors.forHsi()` maps HSI 1 to good, 2 to fair, worse to poor, which is what the contact meter draws.

6.6 `SessionStore`: the session log

`SessionStore` implements `SessionRecorder` with an append-only `sessions.jsonl` in `filesDir`, one line per completed session, written once at `onSessionEnd`. Decisions arrive at 1 Hz and are accumulated in memory; there is no per-decision I/O. Focused time is measured *between transitions* (a FOCUSED edge starts a streak, the next non-FOCUSED decision closes it), not by counting FOCUSED decisions, so a dropped or rejected epoch cannot silently shorten the total. Each closed streak is split across hour boundaries with `Calendar` in local time into the 24-slot `focusedMsByHour`, so a streak over midnight lands in the right buckets. `onDeferred` and `onFlush` just count. A session with no focused time and nothing deferred is discarded rather than logged. Reading tolerates a truncated last line: a crash mid-append should cost one session, not the history. There is no retention limit; only the developer "clear" deletes the file.

`UiPreferences` is a Preferences DataStore named `ui_prefs` holding `onboardingComplete`, `highDemandTask` and `fastCalibration`. It is deliberately separate from C's `NotificationSettings` and B's `ThresholdStore`, which hold domain state.

6.7 `ui/preview`: the demo stack

`DemoStack.kt` holds in-memory stand-ins for A, B and C (`DemoEegSource`, `DemoFocusEngine`, `DemoSuppressor`, `DemoServiceController`) written so the UI could be built and previewed before the real modules existed. Its header says it should be deleted at integration and that if anything in `ui/` has to change when that happens, the contract boundary was drawn in the wrong place. It is no longer wired into the container, but it remains for Compose previews and for `DemoFocusEngineTest`, whose four tests are written against the `FocusEngine` contract (uncalibrated stays UNKNOWN; one spike does not silence the phone; clean separation gives AUC 1.0; empty input gives 0.5) and so should pass against B's pipeline too. `DemoFocusEngine` is also a useful 150-line summary of what B's pipeline does, minus smoothing, the calibration reference, artifact holdoff and drift.

7 Integration: what changed when the branches met

FILES TO READ ALONGSIDE THIS SECTION

`docs/area_specific_changes.md`, `MUSE_CONNECTION_FIXES.md`, `git log` from `3bec20d` to `bf5774e`

The four branches were merged on 3 October and tested on a Galaxy S23 Ultra with the headband `Muse-3973`. Several behaviours changed during that afternoon, and some of the per-workload documents (`SUMMARY_B.md` in particular) predate them. If you own one of these areas, the version in `main` is the one to describe. Table 9 is the list; the reasons are in the sections above where they belong.

Area	Change	Observed cause
A	10 s signal-loss grace, then flush as `HEADSET_REMOVED`, apply UNKNOWN, reset the engine; session ends itself after 120 s without signal; `appliedState` published.	One electrode at HSI 2 for four seconds released the whole queue and toggled DND twice.
A	`start()` refuses to mark itself started without BLE permission; no contact state or epochs until `linkUp`; stale-advertisement rule and 1 s list polling after a drop; `hold()` moved to `onResume`.	A switched-off headband cycled Disconnected and Poor contact; a launch before onboarding never scanned again.
A	`watchForHeadsetRemoved()` deleted.	It counted `StateFlow` emissions, which only happen on change.
A	`runDecisionLoop()` became a `coroutineScope` block so its collectors are children of `loopJob`.	Found while writing this guide: the collectors were siblings on the service scope and survived `onDestroy`.
B	`FocusPipeline` gained `smoothingWindow` (container passes 8) and `useCalibrationReference` (default on); `RollingZScore.score()` added; `Insufficient` now returns `fitted = false`. B's own tests pass unchanged; five smoothing tests added.	With a real fit of $+0.23/+0.07$ the longest run above T_{high} in 11 minutes was 8 s; the rolling window normalised long focus away.
C	Everything rewritten so it runs: lazy, reusable zen rule with a `ZenPolicy`; `FocusModeGate` and `CatchUpPresenter` seams; max-hold timer; one `InboxStyle` summary; listener filters; keyed queue; `SharedPreferences` settings; nine unit tests.	The merged code had an unassigned `lateinit`, registered the rule before access existed, an empty settings file and a TODO where the summary should be.
D	"Use these thresholds" applies the shown values; 2-back never repeats a letter and runs at 1.5 s; reading has "Finished"; fixation dot on the frame clock; "Not enough clean data" result; chart x-axis fixed at 120 s; pill shows `appliedState`; dead switches removed.	Animations off in Developer options froze the dot; fast calibration always returned `Insufficient`.
Shared	`ServiceController.appliedState` added; `DeferredNotification.category` and `CalibrationResult.fitted` added; `allowTwoFactorCodes` removed; app renamed STFU.	

Table 9: Integration changes by area. The source for this table is `docs/area_specific_changes.md`.

Evidence the signal path works. Replaying the 620 epochs logged during an 11-minute reading session through the live rules, with that user's fitted thresholds, gave:

Normalisation	Time focused
Rolling window, no smoothing	0 %
Calibration reference, no smoothing	87 %
Calibration reference, 8-epoch smoothing	98 %

Live afterwards: reading gave a smoothed z of about +0.4; counting backwards from 300 in sevens gave +2.0 with every sample above T_{high}; idly looking at the screen gave +0.1, inside the dead band, and the state stayed NOT_FOCUSED. All three match what the user was doing.

Open decisions the team has not made. Whether the calibration reference should drift slowly (a long EMA) rather than stay fixed until recalibration; whether the muscle ratio of 1.3 rejects real focus; who owns artifact flagging, A's source or B's rejector; and the engagement-check gaps on the 2-back and the single reading question.

7.1 Test inventory

Area	Test file	What it pins down
B	`FocusStateMachineTest`	dwell timing, dead band, no chatter, gap restart, live threshold edit
B	`ArtifactRejectorTest`	the five reasons in order, 2 s inclusive holdoff, relative muscle rule
B	`FocusPipelineTest`	warm-up, acceptance flags, reset; `RollingZScore` semantics
B	`FocusPipelineSmoothingTest`	the real fit 0.23/0.07 is only reachable with smoothing
B	`FocusPipelineCalibrationTest`	apply only if separable, history always, startup load
B	`DriftDetectorTest`	shift and spread rules, sustain count, prompt latches
B	`CalibrationSessionTest`, `...SettleTest`	drop reasons, settle, holdoff across blocks
B	`ThresholdFitterTest`	hysteresis geometry, scale invariance, AUC ties, insufficient
B	`ThresholdStoreTest`	round trip, atomic write, corrupt lines, validation
C	`NotificationControllerTest`	gate on/off, one summary, max hold, dedupe, breakthrough flag
D	`HomeViewModelTest`	blockers, chart window, gaps, manual release, poor contact
D	`NBackScoringTest`, `FormatTest`, `DemoFocusEngineTest`	scoring rules, formatting, engine contract
A	none	the device layer is verified on the phone; see `MUSE_CONNECTION_FIXES.md`

Table 10: Unit tests, all plain JUnit under `app/src/test`. Run with `./gradlew :app:testDebugUnitTest`.

8 Question bank

Answer each from memory first, then check it against the section cited. The questions are ordered roughly from "anyone on the team should get this" to "only the owner would".

Architecture

Q1. What is the one-sentence description of the app, and which Android mechanism does the suppression?

A. While a Muse EEG headset indicates the wearer is concentrating, hold the phone's notifications back and deliver them as one quiet summary when concentration breaks. Suppression is an `AutomaticZenRule` (Do Not Disturb in Priority mode), not cancelling or snoozing (section 5).

Q2. Name the four original interfaces, who implements each, and the two that were added later.

A. `EegSource` (A), `FocusEngine` (B), `Suppressor` (C), `SessionRecorder` (D). Added: `ServiceController` (A) for the master switch and `NotificationPolicy` (C) for

the settings surface (table 1).

Q3. Why is there no dependency-injection framework?

A. Five singletons do not justify the build cost, and a container the team can read on one screen is worth more than annotations. `FocusApp` builds one `AppContainer`; ViewModels reach it through `CreationExtras.container()`.

Q4. What does UNKNOWN mean and what must every consumer do with it?

A. No usable signal: headset off, disconnected or not settled. Every consumer fails open: the suppressor turns the rule off and flushes, the home pill shows "no signal", and notifications are delivered normally.

Q5. Why are thresholds stored as z-scores rather than raw band power?

A. Absolute band power drifts with placement, hair, moisture, headband tension, time of day and caffeine; a raw number calibrated on Tuesday is wrong on Thursday. z-scores are relative to a distribution, and supplying the right distribution becomes the pipeline's job.

Q6. Which thread do LibMuse callbacks arrive on, and where does the decision loop run?

A. LibMuse calls back on the main thread; the decision loop runs in `FocusService`'s `SupervisorJob + Dispatchers.Default` scope. Disk writes hop to `Dispatchers.IO` (fig. 3).

Q7. Why does the home screen show `appliedState` rather than `FocusEngine.state`?

A. They differ during the 10 s signal-loss grace period and when no session runs; the pill must agree with what the DND rule is actually doing.

Workload A

Q8. Why did the headband never appear at first, and what was the fix?

A. The scan permission lacked the `neverForLocation` flag; on Android 12+ scan results are then dropped unless fine location is held, which was capped at SDK 30. One manifest attribute fixed it.

Q9. Why does `MuseEegSource` poll the manager's list every second instead of waiting for the list-changed callback?

A. LibMuse only fires the callback for a headband it has never seen; a known one is silently rediscovered. After a drop or restart the callback never comes.

Q10. What is `staleDiscoveredTime` for?

A. After a drop LibMuse lists the headband for about 30 s with its old discovery time; connecting to that entry while the headband is off just times out. Only an entry rediscovered after the drop is really advertising.

Q11. Explain `mergeFresh`.

A. LibMuse sends `NaN` per channel until its first clean window and then repeats the last clean value while rejecting the current one. Keeping only values that are finite and changed since the previous packet stops stale values posing as fresh data and one `NaN` channel poisoning the mean. First focus value in about 1 s instead of 90.

Q12. Why does `average()` return `NaN` rather than 0 for an empty channel list?

A. Zero produced a plausible finite index from missing data; `NaN` fails `engagementIndex.isFinite()` and the epoch is marked invalid.

Q13. What are the two drop detectors and the reconnect timing?

A. LibMuse's `DISCONNECTED` callback and a watchdog that treats 5 s of packet silence while connected as a drop. Both call `rescanAfterDrop()` with a 2 s delay. The service's own backoff runs 2, 4, 8, 16, 30 s.

Q14. Why does `emitEpoch` return early unless `linkUp`?

A. Deriving state from an empty horseshoe during a reconnect showed "Poor contact" instead of "Connecting".

Q15. What does `ForegroundEegSource` do and why does it ignore `stop()`?

A. It keeps the headset connected while the app is visible: `hold()` on resume, `release()` on stop unless a session is running. Calibration and the service both call `stop()` when they finish; honouring it would drop the headset with the app open.

Q16. Why is the service's `foregroundServiceType connectedDevice`, and how does the UI know whether the service is alive?

A. Android 14+ requires a typed foreground service with the matching permission; a BLE headset is a connected device. The service is unbound; the UI observes companion `StateFlow`s via `AndroidServiceController`, which reset if the process dies.

Q17. Walk through what happens when the headset is taken off mid-session.

A. Contact degrades, status leaves CONNECTED; after 10 s of unusable signal the effective state becomes UNKNOWN: flush as `HEADSET_REMOVED`, `apply(UNKNOWN)` turns the rule off, `focusEngine.reset()`. Backoff reconnects keep trying; after 120 s the service calls `stopSelf()`, which flushes `SERVICE_STOPPED` and ends the session.

Q18. Why did the old `watchForHeadsetRemoved()` never fire?

A. It counted consecutive emissions of a `StateFlow`, which only emits on change, so it never counted five of anything.

Q19. Why is `runDecisionLoop` a `coroutineScope` block, and what went wrong before it was?

A. So its collectors are children of `loopJob` and die with it in `onDestroy`. They used to be launched on the service scope as siblings, so cancelling the loop job cancelled nothing and a stopped session kept driving the engine and `appliedState` (section 3).

Q20. Can the `emgBroadband` artifact flag fire on the Muse source?

A. Effectively no. LibMuse's relative band powers are fractions in $[0, 1]$ and the thresholds are 3.0.

Workload B

Q21. Write down the focus index and say where each band comes from.

A. Pope et al. (1995) engagement index $\beta/(\alpha + \theta)$. Alpha from the temporal pair TP9/TP10 (less eye contamination), beta nominally 13–20 Hz (above that is forehead muscle), theta over all channels.

Q22. List the rejector's checks in order and the holdoff rule.

A. Source invalid, poor contact (any HSI > 1), non-finite, muscle ($\beta > 1.3\times$ the median of the last 60 clean betas, after 10), holdoff (within 2 s inclusive of a rejection). A primary rejection restarts the holdoff; a holdoff rejection does not extend it.

Q23. Why do muscle-rejected epochs still enter the median window?

A. Otherwise a genuine lasting rise in beta would lock the gate forever. A brief clench cannot move a one-minute median; a lasting shift is absorbed.

Q24. Explain the Schmitt trigger with dwell, including what a dead-band value does.

A. Above T_{high} starts an entry run; below T_{low} starts an exit run; in between cancels both runs but keeps the state. Transition when a run has lasted the dwell, measured between timestamps. Defaults $N = 10$ s, $M = 60$ s; first usable sample goes to NOT_FOCUSED (fail open); a gap over 5 s restarts runs.

Q25. At 1 Hz, how many samples does a 5 s dwell take, and why?

A. Six. The run starts at the first qualifying sample ($t = 0$) and `now - start >= 5` is first true at $t = 5$.

Q26. Describe the two normalisation paths and which one is live.

A. With a calibration reference, $z = (\text{smoothed} - \mu_{\text{cal}})/\sigma_{\text{cal}}$. Without one, the rolling z-score over the last 600 raw values after a 30-sample warm-up. The reference path is live by default because a rolling window re-centres on a long focus period and normalises it away.

Q27. Why is the rolling window fed the raw index when the state machine sees the smoothed one?

A. Smoothing shrinks the SD; a smoothed window compared against the calibration reference would look like spread drift. The window exists for the drift detector and as the fallback.

Q28. Why 8-epoch smoothing, and what is the evidence?

A. A single noisy dip resets the entry run, so "sustained above T_{high}" was unreachable: the longest run in 11 minutes of real reading was 8 s. Replaying 620 epochs: 0 % focused with the rolling window, 87 % with the reference, 98 % with reference plus smoothing.

Q29. When does the drift detector suggest recalibration?

A. Reference present and matching `fittedAtMs`, at least 300 window samples, and for 120 consecutive checks either the mean has shifted more than one reference SD or the SD ratio is above 2 or below $\frac{1}{2}$. The pipeline latches the prompt until a separable calibration.

Q30. What does `CalibrationSession.prepare` drop or discard?

A. `MEDIUM/HIGH` blocks with no score, zero attempts or accuracy below 0.5 (never `LOW`); the first $\min(5\,\text{s}, \text{duration}/12)$ of each block; and everything a single fresh `ArtifactRejector` rejects across the whole recording in time order.

Q31. How are the thresholds placed, and how is AUC computed?

A. z-score the whole run (`MEDIUM` included), take the `LOW` and `HIGH` class means, put the midpoint between them, and set $T_{\text{high}}, T_{\text{low}} = m \pm \max(0.15\,d, 0.05)$ where d is the separation. AUC is Mann–Whitney: the fraction of (high, low) pairs where high scores higher, ties counting half.

Q32. What is the difference between a non-separable and an insufficient fit, and what does each do?

A. Insufficient: fewer than 10 usable `LOW` or `HIGH` samples; nothing saved, `fitted = false`, screen says "not enough clean data". Non-separable: AUC < 0.6 or `HIGH` below `LOW`; appended to history as a finding but not applied; thresholds in use unchanged.

Q33. How does `ThresholdStore` persist, and why not JSON?

A. Two `key=value;` text files, `current.txt` (temp-and-rename) and append-only `history.txt`, under a mutex on `Dispatchers.IO`. `org.json` is a stub on the local JVM, and the format is testable without Robolectric and tolerant of unknown keys.

Q34. Why does `updateThresholds` save before applying, and why does `reset()` keep `recalibrationSuggested`?

A. Saving first lets the store's validation refuse an invalid set before it is half-applied. The stale thresholds are still the ones in use next session, so the prompt should stay up until a successful calibration.

Workload C

Q35. Does the app cancel, snooze or re-post notifications?

A. No. It toggles one zen rule, watches the shade with a log-only listener, and posts one summary of its own. Snooze is the unbuilt v2; cancel-and-repost is impossible because an app cannot post on behalf of another package.

Q36. Why a zen rule and not `setInterruptionFilter()`?

A. The rule composes with the user's own DND, is visible in Settings, carries its own `ZenPolicy` for the breakthrough list, and the system restores the previous state when it turns off. The global call would clobber the user's settings.

Q37. Why is the rule created lazily, and why search for an existing one?

A. The container is built before onboarding grants policy access; the earlier code added a new rule on every launch.

Q38. What does `apply()` do for each state, and why in that order?

A. FOCUSED: gate on. Anything else: gate off, then `flush(FOCUS_BROKE)` if the queue is non-empty, so the summary is posted after the rule is off and is not silenced by it.

Q39. Describe the max-hold valve precisely.

A. A `delay(maxHoldMinutes * 60_000)` started when the queue goes from empty to non-empty, not restarted by later arrivals, cancelled if the queue empties. It flushes with `MAX_HOLD_EXCEEDED` and leaves focus mode on. Default 30 min, slider 5–120, zero disables.

Q40. Which notifications never enter the queue?

A. The app's own package, ongoing ones, group summaries, and foreground-service notifications. Anything else while `isFocused`, keyed on `sbn.key` so an in-place update replaces its entry.

Q41. How is the breakthrough list enforced, and what can the app actually see?

A. The platform enforces it through the rule's `ZenPolicy` (`disallowAllSounds` then re-allow alarms, calls, repeat callers, starred messages). The listener only sees `Notification.category`, so it flags calls and alarms; starred-contact and repeat-caller decisions are invisible to it. `allowedPackages` is flagged but not enforced.

Q42. Describe the catch-up summary.

A. One `InboxStyle` notification on channel `catch_up` at `IMPORTANCE_DEFAULT` (sound, no heads-up), title "n notifications while you were focused", subtitle from the flush reason, seven newest lines, "$+k$ more", constant id 2001 so it replaces itself, tapping opens the app.

Q43. What is the role of `BIND_NOTIFICATION_LISTENER_SERVICE`?

A. A signature permission on the listener's manifest entry so only the system can bind it. The user grants listener access in Settings; the app never requests this permission.

Workload D

Q44. Why does `FocusNavHost` latch the start destination and render nothing until it is known?

A. The onboarding preference is read asynchronously; building the graph early would flash Home then Onboarding, and letting the flipped preference reach `startDestination` after onboarding would rebuild the graph under the user.

Q45. When can the master switch be toggled?

A. `canToggle = serviceRunning || blocker == null`, where the blocker is missing required permissions (wins) or no thresholds. A running service can always be turned off.

Q46. Why is the one-second ticker injected into `HomeViewModel`?

A. A `delay` loop inside the ViewModel would hang every test on the virtual clock; tests pass `emptyFlow()`.

Q47. Three design choices in the live chart and the reason for each.

A. Fixed ±3 SD range so calm and volatile signals look different; x-axis sized to the 120-slot capacity so new points enter from the right at a steady pace; rejected epochs break the line so discarded data is not interpolated.

Q48. Which onboarding steps are required, and why can the user skip anyway?

A. Bluetooth and DND. Fail-open applies to setup: a working app that delivers everything beats a dead end. Special-access pages return no result, so everything is re-read on resume.

Q49. Describe one calibration run.

A. 45 s baseline, then nine 60 s blocks (three per condition) in shuffled order with 8 s rests and a 5 s countdown each, waiting for a trustworthy signal between blocks. A `MEDIUM` block is followed by one question; a block under 50 % valid is re-queued up to three times. Then `fitCalibration` and the result screen.

Q50. Why is the reading question asked after the block, and why can only reading end early?

A. Answering is a different task whose epochs would be mislabelled `MEDIUM`; a finished reader is just waiting. Stopping fixation or 2-back early would only mean less data.

Q51. Why does every task use the same dark scaffold and no keyboard?

A. `DESIGN.md` §2 requires matched visual input across conditions; otherwise the comparison measures brightness and layout as much as cognition.

Q52. How does `SessionStore` measure focused time, and why?

A. Between state transitions, not by counting FOCUSED decisions, so a dropped or rejected epoch cannot shorten the total. Streaks are split across hour boundaries into a per-session 24-slot list. Written once per session as a JSONL line.

Q53. Where does each settings control write?

A. Dwell sliders to the engine and B's `ThresholdStore`; max hold and the four toggles to C's `NotificationSettings`; fast calibration and the task choice to D's DataStore. The dwell sliders are hidden until thresholds exist.

Q54. Where would you switch to the fake or OSC source?

A. Nowhere in the UI. Edit the one line in `AppContainer.kt`; the demo-mode switch was removed during integration.